CSE450 Capstone Project Report

ATLAS: An Adaptive Tutoring System using Bayesian Knowledge Tracing over a Prerequisite Skill Graph

Submitted by

Sakif Naieb Raiyan	2105065
Sayaad Muzahid Masfi	2105066
Aurchi Chowdhury	2105083
Omar Abdur Razzaque	2105094
Rubaiyat - E- Zaman	2105104

Supervised by

Dr. Md. Shariful Islam Bhuyan

Associate Professor, Department of CSE, BUET



Department of Computer Science and Engineering

Bangladesh University of Engineering and Technology

Dhaka, Bangladesh

Session 2024–2025

September 2026

CANDIDATES' DECLARATION

We declare that the work presented in this report, titled "ATLAS: An Adaptive Tutoring System using Bayesian Knowledge Tracing over a Prerequisite Skill Graph", is the outcome of the engineering work carried out by us under the supervision named on the title page.

All prior work, third-party code, libraries, data, copyrighted material, and tools used in the project have been properly acknowledged. Any use of generative AI tools has been disclosed. We accept accountability and responsibility for the accuracy and integrity of this report and for our individual contributions to the project.

Sakif Naieb Raiyan

Sakif Naieb Raiyan (2105065)

Muzahid Masfi

Sayaad Muzahid Masfi (2105066)

Aurchi
29.09.26

Aurchi Chowdhury (2105083)

Omar Abdur Razzaque

Omar Abdur Razzaque (2105094)

Rubaiyat

Rubaiyat - E- Zaman (2105104)

ACKNOWLEDGEMENT

We thank our supervisor for the guidance and critical questioning that shaped this project, and in particular for pressing us on the point that ultimately defined the work: that an adaptive tutor is only as good as the knowledge structure underneath it, and that a shallow skill map cannot be rescued by a sophisticated inference algorithm sitting on top of it.

We are grateful to the Department of Computer Science and Engineering, BUET, for the CSE 450 Capstone framework, which gave this work both its scope and its deadlines.

The subject-matter content of this system is drawn from standard Higher Secondary Certificate (HSC) textbooks and past admission-test question banks for Bangladeshi engineering universities. We acknowledge the authors and publishers of that material; it is used here strictly as source material for an academic prototype and is not redistributed as part of the system.

ABSTRACT

Every year a large cohort of Bangladeshi students prepares for the engineering university admission tests (BUET, KUET, RUET, CUET) by working through undifferentiated question banks. A student who repeatedly fails questions on integration by parts is rarely told that the actual gap is algebraic manipulation two levels below; coaching centres cannot diagnose at that granularity at scale, and existing digital platforms largely serve the same fixed content to everyone. The engineering problem is therefore one of inference under uncertainty: from a sparse, noisy sequence of multiple-choice responses, locate a learner's knowledge frontier precisely enough to act on it.

This report presents ATLAS, an adaptive tutoring system that models a learner's latent mastery with Bayesian Knowledge Tracing (BKT) over a directed acyclic graph (DAG) of prerequisite relations between granular skills. Three extensions to textbook BKT carry the design. First, the guess and slip parameters are conditioned on the Bloom's Taxonomy level of the question, so that a correct answer on a recall item and a correct answer on an analysis item update belief by different amounts. Second, a correct response propagates upward through the prerequisite graph, raising every ancestor to at least the descendant's posterior, since demonstrating a dependent skill is evidence for the skills it depends on. Third, a skill's learning-rate parameter is gated conjunctively: it rises only when *all* of its direct prerequisites are mastered, so a skill is not assumed to be learning while a foundation beneath it is missing. An adaptive diagnostic selects each next question by a balanced-volume heuristic over untested ancestors and descendants, at one Bloom level above the learner's current estimated band.

The system was built as a FastAPI backend, a React frontend rendering Bangla text with embedded LaTeX mathematics, and a PostgreSQL (Supabase) store. A retrieval-augmented generation pipeline over six HSC textbooks (4,474 parsed question records) produces the question bank, with a two-model generate-then-verify design in which a second model blind-re-solves each generated item before it is accepted; it has produced 12,223 verified questions covering 89% of the 10,128 (skill, Bloom level) specifications. The knowledge structure itself was the dominant engineering effort: an initial 430-skill ontology derived from a sample of the corpus proved too sparse to support the gating policy, and was rebuilt from the full corpus into 1,688 skills and 1,743 prerequisite edges covering all 183 syllabus topics, validated as an acyclic graph of depth 9 with no isolated skills.

Verification is structural and functional rather than empirical: a 29-check credential-free harness exercises the diagnostic, mastery propagation and prerequisite spillover against an in-memory database stand-in and passes completely; a live end-to-end check drives every flow in all 51 sections against the production question bank; and the ontology passes an automated structural validator with zero errors. The live check also exposed defects that remain open, among them that the ancestor ordering is not kept through wrong answers and that selection is slow where the bank is thin. The system has not yet been evaluated with human learners, and establishing that its mastery estimates predict real performance remains the principal item of future work.

Keywords: Bayesian Knowledge Tracing; adaptive learning; prerequisite graph; Bloom's Taxonomy; intelligent tutoring systems; retrieval-augmented question generation

Contents

Candidates' Declaration i

Acknowledgement ii

Abstract iii

1 Problem Identification and Formulation 1

1.1 Background and Motivation 1

1.2 Problem Statement 2

1.3 Complexity of the Problem 3

1.4 Project Objectives and Scope 6

1.4.1 Objectives 6

1.4.2 Scope 6

2 Preliminary Research 8

2.1 Literature Review 8

2.1.1 Modelling what a learner knows 8

2.1.2 Cognitive level as a modelling variable 9

2.1.3 Automatic generation of assessment items 9

2.1.4 What the literature establishes, and what it leaves open 9

2.2 Existing Solutions and Technologies 10

2.3 Market and User Research 11

2.4 Feasibility Study 12

3 Requirements Analysis 13

3.1 Project Objectives and Constraints 13

3.1.1 Stakeholders and their expectations 13
3.1.2 Functional requirements 13
3.1.3 Non-functional requirements 15
3.1.4 Constraints 16
3.2 Regulatory Requirements and Standards 16
3.3 Formulation of Solutions 16

4 System Design and Implementation **19**
4.1 Functional Design and Sub-problem Partitioning 19
4.2 Design Refinement and Simulation 20
4.2.1 The knowledge-tracing model 20
4.2.2 Conditioning the noise parameters on cognitive level 22
4.2.3 The three-phase mastery update policy 23
4.2.4 Adaptive question selection 25
4.2.5 Prerequisite spillover 28
4.2.6 The skill graph, and its rebuild 28
4.2.7 Data model 31
4.2.8 API surface 31
4.2.9 Content generation pipeline 32
4.2.10 Design calculations 34
4.3 Application of Tools 35
4.4 Implementation 36
4.5 Deployment 37

5 Investigation and Evaluation **39**
5.1 Experimental Investigation 39
5.1.1 Investigation 1: is local model inference viable for content generation? . 39
5.1.2 Investigation 2: is the ontology's granularity sufficient to support the gating policy? 40

5.1.3 Investigation 3: can prerequisite edges be proposed mechanically? . . . 41
5.1.4 Investigation 4: do automated quality heuristics over-flag? 41
5.2 Performance Evaluation 42
5.2.1 Setup and acceptance criteria 42
5.2.2 Results 42
5.2.3 Live end-to-end check 45
5.2.4 Evaluation against requirements 47
5.2.5 Evaluation against objectives 49
5.2.6 Limitations of this evaluation 49
5.3 Deviations and Design Revisions 50
5.3.1 Topic filter matched no rows (found, analysed, fixed) 50
5.3.2 Session state is not persistent 51
5.3.3 Authentication is username-only 51
5.3.4 Ontology adoption, and the coverage debt it exposes 51
5.3.5 Skill identifiers truncated at 1,000 rows (found, fixed) 52
5.3.6 Diagnostic ignored its section and stored mastery (found, fixed) 52
5.3.7 Answer key readable by the client (found, fixed) 52
5.3.8 Ancestor ordering not kept through wrong answers (found, open) 53
5.3.9 Fallback items credited to the target skill (found, open) 53
5.3.10 Diagnostic length overstated (found, open) 54
5.3.11 Sections unlocked by another section's diagnostic (found, open) 54
5.3.12 Rendering defects in generated content (found, open) 55
5.3.13 Ontology mirror in the database is stale (found, open) 55

6 Project Management, Teamwork, and Impact 56

6.1 Project Management and Finance 56
6.1.1 Planning and Risk Management 56
6.1.2 Budgeting and Resource Identification 57
6.1.3 Economic Analysis and Financial Prospecting 59

6.1.4 Sustainability in Business and Commercialisation 60
6.2 Individual and Teamwork 60
6.2.1 Participation and Contribution 60
6.2.2 Collaboration and Conflict Resolution 60
6.2.3 Leadership and Direction 61
6.2.4 Multidisciplinary Engagement 62
6.3 Impact on Society 62
6.4 Environmental Impact and Life Cycle Sustainability 63
6.5 Ethics 64
6.5.1 Equity and Inclusivity 64
6.5.2 Accountability and Personal Responsibility 64
6.5.3 Proper Use of Intellectual Property 65
6.5.4 Professionalism and Ethical Codes 65

7 Conclusion **67**
7.1 Conclusion and Future Work 67
7.1.1 Achievement against objectives 67
7.1.2 Limitations 68
7.1.3 Future work 68
7.2 Continuous Engagement and Self-directed Learning 69

References **71**

List of Figures

1.1 One wrong answer, four different deficits 2

1.2 The problem as a closed inference–selection loop 3

2.1 The gap no surveyed solution fills 11

3.1 Use cases mapped to functional requirements 15

4.1 The learner's journey through ATLAS 19

4.2 System architecture. Solid boxes are the online serving path; dashed boxes are offline processes that run before serving and write their artefacts into the compiled ontology and the question bank. 20

4.3 BKT as a two-state hidden Markov model 21

4.4 Mastery bands and the one-level-up selection rule 23

4.5 Effect of the Bloom level on a single update 23

4.6 The three-phase mastery update 24

4.7 The policy on the integration-by-parts fragment 25

4.8 The conjunctive gate in practice 26

4.9 Balanced-volume scores for a real section 26

4.10 The question-search fallback ladder 27

4.11 Prerequisite spillover rules 28

4.12 A fragment of the skill graph, illustrating the granularity target. "Integration by parts" is not one node but a chain: recognising applicability, choosing the decomposition, and executing it — each separately testable, and each with its own prerequisites, so that a failure can be localised to the step that actually broke. 29

4.13 How the skill graph was built 29

4.14 Shape of the served skill graph 30

4.15 Database schema with foreign keys and row counts 32

4.16 One diagnostic, request by request 33

4.17 The question-generation pipeline 33

4.18 Question-bank coverage of the served ontology 35

5.1 Mechanical heuristics against hand review 42

5.2 A live diagnostic in the Ideal Gas section 46

5.3 Live topic practice with spillover 46

5.4 Responsiveness measured in the live check 47

5.5 Position of the correct option, stored and served 53

5.6 Ancestor-ordering inversions by answer pattern 54

6.1 Project schedule. The ontology rebuild in September was not in the original plan; it was forced by Investigation 2 (Section 5.1). 57

6.2 Version-control activity by week 57

List of Tables

1.1 Complex engineering problem characteristics (P1–P7) and evidence from this project 4

1.2 Knowledge profiles referenced by P1 5

2.1 Existing solutions surveyed, and their limitations for this problem 10

3.1 Stakeholders, expectations, and conflicts 13

3.2 Functional requirements 14

3.3 Non-functional requirements 15

3.4 Standards and regulatory requirements, and how the design responds 17

3.5 Evaluation of candidate solutions against criteria drawn from the requirements . 18

4.1 Subsystem responsibilities 21

4.2 Bloom-conditioned guess and slip parameters, and the mastery band each level occupies 22

4.3 Legacy ontology versus full-corpus rebuild 30

4.4 Rebuilt ontology by subject 30

4.5 Database schema 31

4.6 API endpoints 34

4.7 Tool selection, justification, and relevant limitations 35

5.1 Structural effect of the ontology rebuild and its editorial pass 40

5.2 End-to-end verification harness: checks and what each establishes 43

5.3 Live end-to-end check against the production question bank (51 sections, 4,862 requests, 14,066 assertions) 45

5.4 Requirements traceability and status 48

6.1 Risk register: principal risks, their realisation, and mitigation 58

6.2 Development resources: actual and notional cost 58

6.3 Indicative operating cost for a production deployment (order-of-magnitude estimate, not a quotation) 59

6.4 Contribution by subsystem 61

7.1 Achievement against the objectives of Section 1.4 67

Chapter 1

Problem Identification and Formulation

1.1 Background and Motivation

Admission to the engineering universities of Bangladesh — BUET, KUET, RUET and CUET — is decided by a single competitive examination taken after the Higher Secondary Certificate (HSC). The test covers Mathematics, Physics and Chemistry, and the ratio of candidates to available seats is severe enough that preparation is effectively a year-long full-time activity for those who attempt it. The dominant preparation method is volume: a student buys one or more question banks compiled from previous years' papers and works through them, usually alongside coaching-centre classes that move at a fixed pace through a fixed syllabus.

This method has a structural weakness that has nothing to do with effort. When a student fails a question, the question bank tells them the correct answer; it does not tell them *why* they failed. A wrong answer on a problem requiring integration by parts may indicate that the student does not recognise when the technique applies, or cannot choose u and dv correctly, or chose them correctly but made an algebraic error in the resulting simpler integral, or does not know the derivative of the function involved. These are four different deficits requiring four different remedies, and they are invisible in a score. A human tutor localises them by watching the student work — which is precisely what one-to-one tutoring is effective at, and precisely what does not scale to a national cohort [1]. Figure 1.1 makes the point concrete.

The consequence is misallocated study time. A student who does not know which of their foundations is weak has no rational basis for choosing what to study next, and defaults to either linear syllabus coverage or to practising the topics where failure is most visible — often the topics *downstream* of the actual gap. Time spent re-attempting integration problems does not repair an algebra deficit; it merely produces repeated failure on the same prerequisite, which is both inefficient and demoralising.

What is needed is a system that infers, from the student's own answers, where in the structure of the subject their knowledge stops — and then selects the next question at that frontier rather than at the point in the syllabus the calendar happens to have reached.

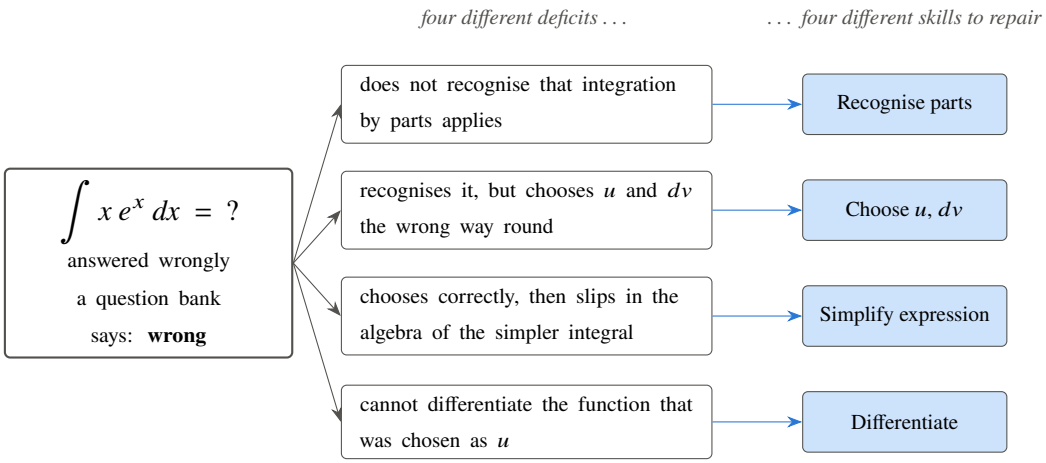


Figure 1.1: Why a score cannot direct study. One wrong answer to one integration question is consistent with four different deficits, each located at a different skill and each needing a different remedy. The same answer sheet looks identical in all four cases, which is why localisation has to come from a model of the prerequisite structure rather than from the score.

1.2 Problem Statement

Problem. Given a learner and a sequence of their responses to multiple-choice questions, estimate the learner's mastery of every skill in a subject domain, and select the next question so as to localise and repair the learner's knowledge frontier.

Formulated in engineering terms:

- **Entities.** A set of *skills* $S = \{s_1, \ldots, s_n\}$, each an atomic, independently testable unit of knowledge or procedure, annotated with one Bloom's Taxonomy cognitive level. A set of directed *prerequisite* edges $E \subseteq S \times S$, where $(s_i, s_j) \in E$ means s_i must be held before s_j is reachable. A bank of questions, each tagged with the skill it tests, its Bloom level and its topic. A set of learners.
- **Inputs.** For a learner u, an observed sequence of binary outcomes $(q_1, c_1), (q_2, c_2), \ldots$ where q_t is the question served at step t and $c_t \in \{\text{correct}, \text{incorrect}\}$.
- **Latent state.** For each skill, $P(L_{u,s}) \in [0, 1]$, the probability that learner u has learned skill s. This is never observed directly; it must be inferred from noisy evidence.
- **Outputs.** (i) The posterior $P(L_{u,s})$ for all $s \in S$ after each response; (ii) the identity of the next question to serve.
- **Quantities to be guaranteed.** The graph (S, E) must be acyclic, or the notion of "prerequisite" is incoherent and propagation does not terminate. Mastery estimates must respect the graph: no skill may be credited as mastered while an ancestor it depends on is

estimated as unmastered. Question selection must terminate even when the bank has no item matching the ideal specification.

- **Assumptions.** Responses are conditionally independent given the latent state; a learner does not forget within a session (no decay term); each question tests primarily one skill; the prerequisite structure is the same for all learners.

Figure 1.2 draws this formulation as the loop the system runs on every response.

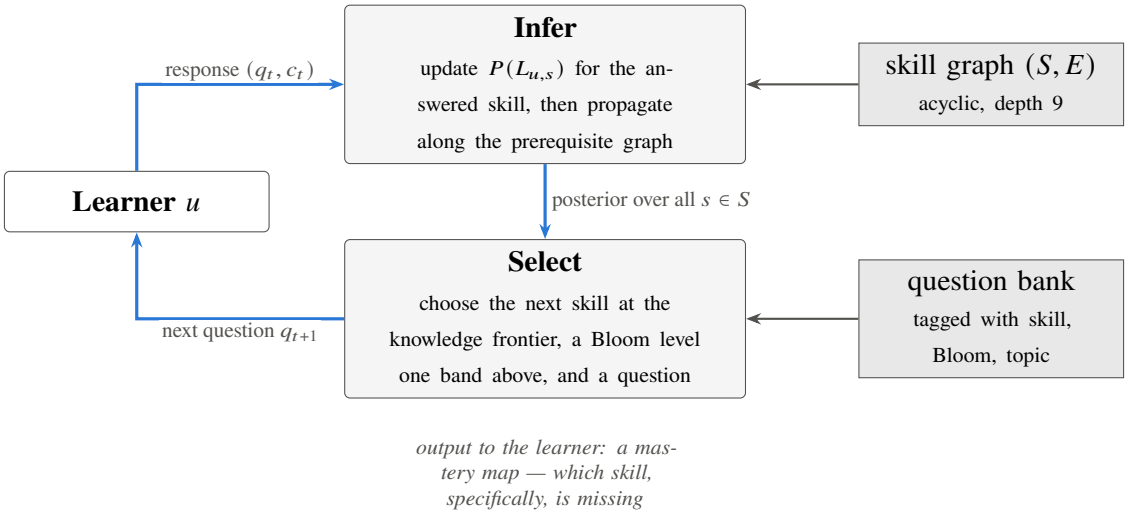


Figure 1.2: The problem as a closed loop. Each response is weak, noisy evidence about one skill; the inference step turns it into a posterior over every skill in the graph, and the selection step spends the next question where that posterior is least certain. The two stores are the fixed inputs defined in Section 1.2.

Engineering knowledge demanded. The formulation above is not solvable by routine application of standard components. It requires: probability theory and Bayesian inference (the latent-state update); graph theory (acyclicity checking, topological ordering, transitive reduction, ancestor traversal); algorithm design (the question-selection heuristic and its fallback ladder); database design under referential constraints; the pedagogical literature on knowledge tracing and cognitive taxonomies, which supplies the model family and the level annotation scheme; and, for the content pipeline, applied natural language processing — retrieval-augmented generation, embedding-based retrieval, and the validation of machine-generated assessment items.

1.3 Complexity of the Problem

A complex engineering problem must have characteristic P1 and some or all of P2–P7 (Table 1.1). P1 requires in-depth knowledge at the level of one or more of the knowledge profiles K3–K8 listed in Table 1.2.

Table 1.1: Complex engineering problem characteristics (P1–P7) and evidence from this project

Code	Definition	Evidence from this project
P1	**Depth of knowledge required.** Cannot be resolved without in-depth engineering knowledge at the level of one or more of K3, K4, K5, K6, or K8, which allows a fundamentals-based, first-principles analytical approach.	**K3, K4, K5, K8.** K3: the Bayesian update is derived from first principles, not called from a library — the posterior and transition equations are implemented directly (Section 4.2.1). K4: knowledge tracing and cognitive-taxonomy literature supplies the model family [2, 3]. K5: the DAG, its acyclicity guarantee and transitive reduction are what make the gating policy implementable. K8: the design engages directly with the research literature on BKT variants [4–6] and on prerequisite-structured student models [7].
P2	**Range of conflicting requirements.** Involves wide-ranging or conflicting technical, engineering, and other issues.	Granularity conflicts with question-bank coverage: finer skills localise gaps better but multiply the items needed (the rebuilt ontology raised the generation space from 430×6 to $1{,}688 \times 6$ tuples). Diagnostic length conflicts with learner patience: a diagnostic short enough to be sat through cannot directly measure 1,688 skills, forcing inference by propagation rather than measurement. Adaptivity conflicts with stability: an aggressive learning rate tracks change quickly but overreacts to a lucky guess. Section 3.3 resolves these explicitly.
P3	**Depth of analysis required.** Has no obvious solution and requires abstract thinking and originality in analysis to formulate suitable models.	Textbook BKT is per-skill and independent; it cannot express "mastery of s_j is impossible while s_i is unmastered." Making it graph-aware required an original three-phase policy (Bloom-conditioned update, ancestor pull-up, conjunctive transition gating) that is not a published algorithm but a composition designed for this problem (Section 4.2.3).
P4	**Familiarity of issues.** Involves infrequently encountered issues.	Bangla-language STEM content with embedded LaTeX mathematics is outside the well-trodden path: it broke JSON escaping (a corruption affecting a measured 13.4% of one generated batch), broke text rendering in the browser, and is poorly served by English-centric embedding models.

continued on next page

Table 1.1 — continued from previous page

Code	Definition	Evidence from this project
P5	**Extent of applicable codes.** Is outside problems encompassed by standards and codes of practice for professional engineering.	No standard governs what a "skill" is, how finely a syllabus should be decomposed, or when an automatically generated assessment item is fit to serve to a student. These decisions were made against explicit project-defined criteria (Section 3.2) because no code of practice supplies them.
P6	**Extent of stakeholder involvement and conflicting requirements.** Involves diverse groups of stakeholders with widely varying needs.	Learners want the shortest path to a higher score; teachers want syllabus fidelity and correct content; the system's own policy wants maximal information per question. These pull in different directions — the most informative question is often not the most encouraging one. Section 3.1 tabulates the conflicts.
P7	**Interdependence.** Is a high-level problem including many component parts or sub-problems.	Five interdependent subsystems: the knowledge ontology, the inference engine, the question bank and its generation pipeline, the serving API, and the learner-facing interface. The ontology is upstream of all others — its granularity determines what the engine can infer and what the generator must produce, which is why a sparse first ontology forced a full rebuild (Section 4.2.6).

Table 1.2: Knowledge profiles referenced by P1

Code	Description
K3	A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline.
K4	Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.
K5	Knowledge that supports engineering design in a practice area.
K6	Knowledge of engineering practice (technology) in the practice areas in the engineering discipline.
K8	Engagement with selected knowledge in the research literature of the discipline.

1.4 Project Objectives and Scope

1.4.1 Objectives

The project set the following verifiable objectives. Chapter 5 evaluates the delivered system against each.

O1. Construct a prerequisite skill graph for the HSC Mathematics, Physics and Chemistry syllabus at a granularity fine enough that a single node names one independently testable action, and verify that the graph is acyclic and free of isolated nodes.

O2. Implement a knowledge-tracing engine that maintains a per-learner, per-skill mastery estimate, updates it from each response by Bayesian inference, and conditions that inference on the cognitive level of the question.

O3. Make the estimate graph-aware, such that evidence propagates to prerequisite skills and no skill can be credited above an unmastered prerequisite.

O4. Deliver an adaptive diagnostic that localises a learner's frontier within a bounded number of questions (target 30) rather than testing every skill.

O5. Deliver targeted practice that drives a chosen topic to a defined mastery threshold and detects when repeated failure is caused by a missing prerequisite rather than by the topic itself.

O6. Produce a question bank aligned to the skill graph, in Bangla with correct mathematical typesetting, via an automated pipeline with a verification stage.

O7. Deliver a usable web application exposing the diagnostic, practice and a mastery view to a learner.

O8. Verify the engine end to end with an automated test harness that runs without external credentials.

1.4.2 Scope

Included: the three admission-test subjects; multiple-choice items with four options; a single-process web deployment sufficient for demonstration; Bangla content with LaTeX mathematics; English and Bangla interface text; and an administrator view for curating the skill graph and for cohort-level mastery statistics with data export.

Explicitly excluded: written/subjective answer grading; handwriting or image input; mobile-native applications; per-class teacher dashboards and classroom management; multi-tenant

institutional deployment; live proctoring; and any claim of improved examination outcomes, which would require a controlled study with human participants that this project did not undertake (Section 5.2.6).

Chapter 2

Preliminary Research

2.1 Literature Review

2.1.1 Modelling what a learner knows

The problem of inferring an unobservable knowledge state from observable responses has a long history in intelligent tutoring systems. Bayesian Knowledge Tracing (BKT), introduced by Corbett and Anderson [2], remains the reference formulation: a skill's mastery is a hidden binary variable, and four parameters govern its evolution — a prior $P(L_0)$, a learning rate $P(T)$, and two noise terms, the slip probability $P(S)$ (knowing but answering wrongly) and the guess probability $P(G)$ (not knowing but answering correctly); Figure 4.3 in Chapter 4 draws it as a two-state model. The model's durability comes from the fact that these four parameters are interpretable to a teacher, which is not true of most of its successors.

The literature has extended BKT along three axes that are directly relevant here. The first is *parameter individualisation*: Yudelson et al. [5] fit per-student parameters and report improved predictive accuracy, establishing that treating all learners as identical is a real source of error. The second is *contextualisation of noise*: Baker et al. [4] estimate slip and guess per response rather than per skill, on the argument that a guess on a four-option item is not the same event as a guess on an open response. Pardos and Heffernan [6] make the related argument for item difficulty. The third is *structure between skills*: Käser et al. [7] model students with dynamic Bayesian networks in which skills are linked by prerequisite relations, and report that exploiting that structure improves prediction over independent per-skill models.

Competing model families exist. Learning Factors Analysis [8] and Performance Factors Analysis [9] replace the hidden-state formulation with logistic regression over counts of prior successes and failures. Deep Knowledge Tracing [10] replaces it with a recurrent neural network and reports substantial accuracy gains on large datasets. These are relevant as alternatives considered (Section 3.3) but share a property that disqualified them here: their internal state is not interpretable as "this student has not mastered this specific skill," which is exactly the output this system must produce in order to act on it.

A separate and older tradition, knowledge space theory [11, 12], formalises a domain as a partially ordered set of knowledge states and asks which questions most efficiently locate a learner within it. This is the theoretical ancestor of the prerequisite graph used here, and it supplies the central insight that a well-structured domain permits assessment far shorter than the number of skills, because observing one response constrains many states at once.

2.1.2 Cognitive level as a modelling variable

Bloom's taxonomy [13], in its revised form [3], classifies learning objectives into six ascending cognitive levels from Remember to Create. Its use in tutoring systems is usually editorial — a way of labelling items — rather than a variable inside the student model. The observation this project builds on is that the taxonomy has a direct probabilistic reading: a four-option recall question can be answered correctly by chance far more easily than an analysis question, and a learner who genuinely knows an analysis-level skill is more likely to slip on it through a procedural error. That makes the Bloom level a natural conditioning variable for $P(G)$ and $P(S)$, which is the refinement Baker et al. [4] argue for on empirical grounds but implemented from a pedagogical prior rather than a fitted one.

Vygotsky's zone of proximal development [14] supplies the selection rule: instruction is most effective just beyond what the learner can already do unaided. Translated into this system's terms, the next question should sit one Bloom level above the learner's current estimated band — which is the rule the diagnostic implements.

2.1.3 Automatic generation of assessment items

Kurdi et al. [15] survey automatic question generation for education and report the field's persistent difficulty: generated items are often syntactically fine but pedagogically hollow, and evaluation of their quality is usually thin. Retrieval-augmented generation [16] addresses part of this by grounding generation in retrieved source material rather than in model memory, which both improves factual accuracy and ties items to a specific curriculum. Multilingual sentence embeddings [17] make such retrieval feasible over a non-English corpus.

2.1.4 What the literature establishes, and what it leaves open

Established: BKT is an adequate and interpretable model of per-skill mastery; its noise parameters should not be uniform across items; prerequisite structure between skills carries real predictive value; and short adaptive assessment is possible when the domain is well structured.

Left open, and therefore the space this project works in: the literature provides no standard

method for *constructing* a fine-grained prerequisite graph for a specific national syllabus — the graphs in published work are either small, hand-built by domain experts, or learned from response data that does not exist before a system is deployed. Nor does it settle how a Bloom-conditioned, graph-propagating variant of BKT should behave when the question bank is incomplete, which is the normal condition of a system built before it has users.

2.2 Existing Solutions and Technologies

Table 2.1: Existing solutions surveyed, and their limitations for this problem

Category	Examples	Strengths and limitations for this problem
Adaptive commercial platforms	ALEKS; Knewton-style adaptive engines	Strength: mature adaptive engines with knowledge-space foundations and demonstrated scale. Limitation: closed, licensed per seat, and built around Western curricula — no HSC syllabus coverage, no Bangla content, and no ability to add either.
Bangladeshi ed-tech platforms	10 Minute School, Shikho and similar	Strength: strong local content, Bangla-first, priced for the local market, already trusted by the target cohort. Limitation: predominantly content-delivery and fixed mock tests; assessment reports a score per test or per chapter rather than inferring a per-skill knowledge state, so the student still has to decide what to study next.
Generic quiz and spaced-repetition tools	Quizlet, Anki, Google Forms	Strength: free, simple, and effective for pure recall [18]. Limitation: no model of prerequisite structure at all — a card is either due or not due — so they cannot diagnose *why* an item was failed or redirect to an upstream deficit.
Printed question banks and coaching centres	The incumbent method	Strength: authoritative content, human explanation, peer environment. Limitation: fixed pace for a heterogeneous room; diagnosis at the granularity of a chapter, not a skill; and the marginal cost of one-to-one attention is precisely what prevents it scaling [1].
Open-source ITS frameworks	pyBKT and similar research toolkits	Strength: reference implementations of the model family, useful for validation. Limitation: libraries, not products — they supply the inference but neither the ontology, the content, nor the serving layer, which is where the majority of this project's effort actually went.

The gap. No surveyed system combines (i) a fine-grained prerequisite model of *this* syllabus,

(ii) inference that is interpretable per skill and propagates across prerequisite structure, and (iii) Bangla content with correct mathematical typesetting. Platforms that have the content lack the model; platforms that have the model lack the content and the language. ATLAS targets that intersection (Figure 2.1).

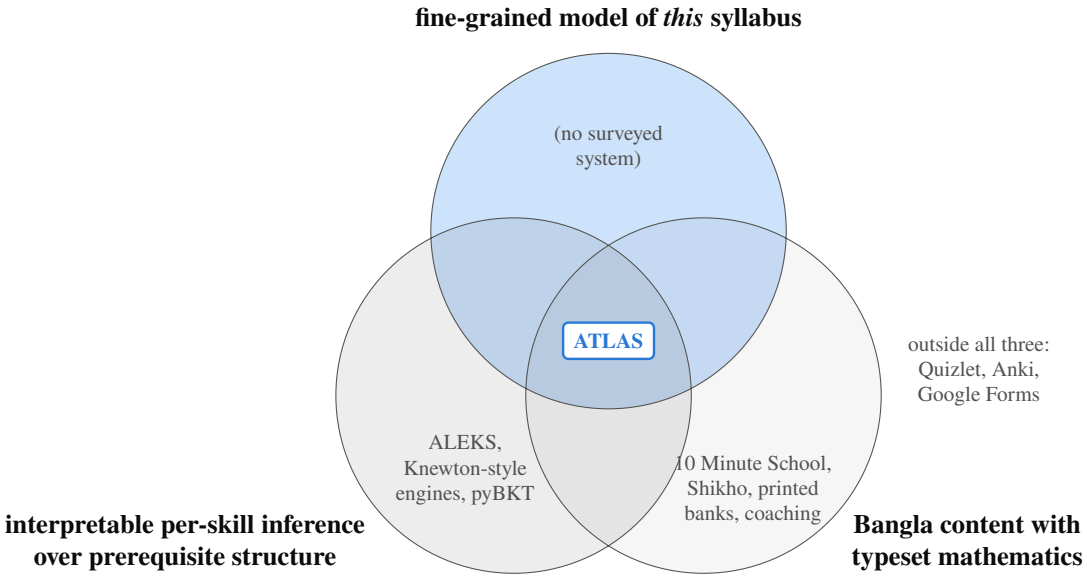


Figure 2.1: The gap identified in Section 2.2. Each surveyed category (Table 2.1) covers at most one of the three capabilities the problem needs: platforms with the content lack the model, platforms with the model lack the content and the language, and no system models the HSC syllabus at skill granularity. ATLAS targets the intersection.

2.3 Market and User Research

User base. The primary users are HSC-completing students in Bangladesh preparing for engineering admission tests, typically in a concentrated period between the HSC examination and the admission tests. Secondary users are repeat candidates and students in the HSC years using the system for syllabus practice rather than test preparation.

What users do instead. Current practice, established from the team's own experience of the same examination and from the structure of the material that dominates the market, is: solve past-paper compilations; attend coaching classes; sit periodic mock tests that return a rank and a score. The gap this leaves is diagnostic, not informational — students do not lack content, they lack a reliable answer to "what should I study next?"

Demand characteristics. Willingness to spend on preparation is demonstrably high, since coaching enrolment is close to universal among serious candidates, but willingness to spend on *software* is low; the realistic price point for a digital tool is at or near zero for the student, with revenue arriving through institutions if at all (Section 6.1.3).

Evidence and its limits. The characterisation above is grounded in the team's direct familiarity with the examination and an informal survey of available platforms and printed material. It is *not* grounded in a structured survey of prospective users with a defined sample; no such study was conducted. This is a genuine weakness of the project's user research, stated here rather than obscured, and Section 7.1 lists it as future work.

2.4 Feasibility Study

Technical. Feasible, and demonstrated. The inference model is computationally trivial (a constant-time update per response plus an ancestor traversal bounded by graph depth, measured at 9). The content pipeline was the real technical risk: it depends on a large language model producing correct Bangla mathematical content at scale. That risk was retired empirically — an initial attempt using local models on CPU hardware proved infeasible on throughput grounds and was abandoned, and a hosted API with a generate-then-verify design was substituted successfully (Section 4.2.9).

Economic. Feasible at prototype scale at essentially zero marginal cost: every service used (database, model API, hosting) has a free tier adequate for development and demonstration. It is *not* feasible to assume those free tiers extend to production; Section 6.1.3 models the real operating cost.

Operational. The adoption question is the binding one. The system asks a student to answer a diagnostic of up to 30 questions (sized to the section, between 5 and 30) before it gives them anything, which is a real cost to a user under time pressure who is accustomed to immediate practice. Mitigation is to make the diagnostic itself feel like practice — it serves genuine past-paper-style questions, not artificial calibration items.

Legal and regulatory. Feasible with attention. The system stores learner performance data, which is personal data; the source content is drawn from copyrighted textbooks and question banks, which constrains redistribution. Both are addressed in Section 3.2 and Section 6.5.3.

Conclusion. The project is feasible, and the principal assumption it depends on is that a machine-constructed skill ontology, validated structurally but not yet by domain experts, is an adequate foundation for the inference built on top of it. That assumption is explicitly flagged as unretired (Section 5.2.6).

Chapter 3

Requirements Analysis

3.1 Project Objectives and Constraints

3.1.1 Stakeholders and their expectations

Table 3.1: Stakeholders, expectations, and conflicts

Stakeholder	Expectation	Conflict with other stakeholders
Admission candidate (primary user)	Shortest path to a higher score; immediate practice; questions that feel like the real examination; no wasted time.	Wants to start practising at once; the system wants a diagnostic of up to 30 questions first. Wants encouragement; the most informative question is the one they are about 50–70% likely to fail.
Teacher / coaching instructor	Syllabus fidelity, factually correct content, results that map to chapters they teach.	Thinks in chapters; the engine thinks in skills that cut across chapters. Requires content correctness guarantees that a generative pipeline cannot fully give.
Project team (developer)	A system that is demonstrably correct, maintainable, and finishable within one academic session.	Correctness argues for hand-authored content and expert-reviewed ontology; the schedule argues for automation.
Supervisor / examiner	Evidence of engineering depth, rigour, and honest evaluation.	An honest evaluation reports unvalidated components, which conflicts with the team's interest in presenting a finished product.
Content rights holders	Their material not redistributed.	The system needs that material as source for retrieval and generation.

3.1.2 Functional requirements

Figure 3.1 shows the same requirements from the side of the two actors who use the system.

Table 3.2: Functional requirements

ID	Requirement	Objective
FR1	A learner can create an account and sign in.	O7
FR2	The system presents a catalogue of courses, sections and topics derived from the skill ontology.	O1, O7
FR3	A learner can start a diagnostic for a section; the system serves questions one at a time, adaptively selected.	O4
FR4	Each response updates the learner's mastery estimate for the tested skill using Bayesian inference conditioned on the question's Bloom level.	O2
FR5	A correct response propagates upward, raising ancestor skills to at least the descendant's posterior.	O3
FR6	A skill's learning rate is gated on all of its direct prerequisites being mastered.	O3
FR7	Mastery views are locked until the section diagnostic is complete.	O4
FR8	After the diagnostic, a learner can practise a chosen topic until every skill in it reaches the mastery threshold.	O5
FR9	Repeated failure attributable to a missing prerequisite triggers injection of prerequisite-focused questions.	O5
FR10	A learner can view their mastery as both a graph and a table, with subject and topic labels.	O7
FR11	A learner can reset a section and retake its diagnostic.	O7
FR12	Questions render Bangla text with embedded LaTeX mathematics correctly.	O6, O7
FR13	The question bank is generated from the source corpus and aligned to skill, Bloom level and topic.	O6
FR14	The interface is available in English and Bangla.	O7
FR15	An administrator can curate the skill graph: add, edit and delete skills and prerequisite edges, with every edit checked for cycles.	O1
FR16	An administrator can view cohort-level mastery statistics and export all per-skill performance data as CSV or JSON.	O7

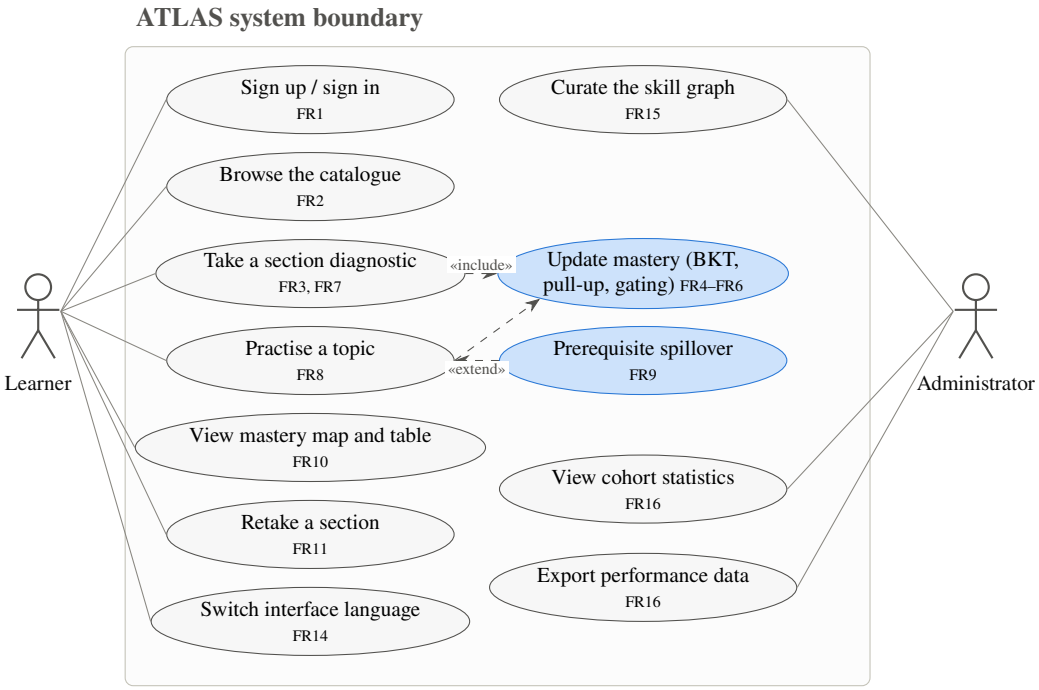


Figure 3.1: Use cases, labelled with the functional requirements of Table 3.2 they realise. The shaded cases are not requested by the learner directly: every diagnostic and practice answer triggers the mastery update, and spillover extends practice when repeated failure points at a missing prerequisite. The administrator cases were added late in the project (FR15, FR16).

3.1.3 Non-functional requirements

Table 3.3: Non-functional requirements

ID	Attribute	Requirement
NFR1	Correctness	The prerequisite graph must be acyclic; mastery estimates must never place an ancestor below its descendant; these must be checked automatically, not assumed.
NFR2	Responsiveness	Next-question selection must complete fast enough to feel immediate in a browser session; the search must terminate even when no ideal item exists.
NFR3	Robustness	Absence of a matching question must degrade gracefully through a defined fallback ladder rather than failing the session.
NFR4	Maintainability	The ontology must be compiled from a single source of truth by a reproducible build, not hand-edited in its served form.
NFR5	Verifiability	The engine must be testable end to end without database credentials.
NFR6	Portability	The system must run on a standard developer machine with no GPU.
NFR7	Localisation	All learner-facing content must support Bangla, including mathematics.
NFR8	Security	Secrets must never enter version control; the database service key must remain server-side.

3.1.4 Constraints

Time: one academic session, with the team also carrying a full course load. **Budget:** effectively zero; every external service had to have a usable free tier (Section 6.1.2). **Hardware:** development on personal laptops with no GPU — this constraint alone eliminated the original local-model content-generation approach. **Data:** no existing learner response data was available, so no model parameter could be fitted empirically; all parameters had to be set from pedagogical priors. **Platform:** web, to avoid the cost of maintaining native clients. **Deployment:** a single process sufficient for demonstration, which is why session state is held in memory (Section 5.3.2).

3.2 Regulatory Requirements and Standards

No single code of practice governs an adaptive tutoring system, so the obligations that do apply were collected from data protection, web security, accessibility, copyright and professional ethics. Table 3.4 lists each one, why it applies to ATLAS, how the design responds, and the gaps that remain.

3.3 Formulation of Solutions

Three genuinely different architectures could each satisfy most of the functional requirements. They differ in where the intelligence lives.

Alternative A — Item Response Theory over a flat item pool. Model each *question* with a difficulty parameter and each learner with a single latent ability parameter; select the next item to maximise information about that ability. This is the classical computerised-adaptive-testing design.

Alternative B — Deep Knowledge Tracing. Model the learner's entire response history with a recurrent network that predicts the probability of answering the next item correctly [10]; select items to maximise learning gain under the network's predictions.

Alternative C — Bayesian Knowledge Tracing over a prerequisite DAG (selected). Model each skill separately with an interpretable four-parameter hidden-state model, link skills by prerequisite edges, and propagate evidence along those edges.

Table 3.4: Standards and regulatory requirements, and how the design responds

Instrument	Why it applies	How the design complies
Data-protection obligations for personal data of minors and young adults [19]	The system stores per-learner performance data; many users are 17–19 years old.	Data minimisation: the schema stores only a username and per-skill mastery — no email, phone, address, date of birth or payment data. No third-party analytics or advertising trackers are embedded. Deletion is supported at section granularity by the retake endpoint. *Gap:* the current prototype has no password authentication, so this minimal data is not access-controlled; see Section 5.3.3.
OWASP Top 10 [20]	The system is a public-facing web application with a database behind it.	All database access goes through a parameterised client library rather than string-concatenated SQL, addressing injection. The privileged service key is held server-side in an environment file that is git-ignored and never shipped to the browser, addressing sensitive-data exposure. *Gap:* broken access control — username-only login and wide-open CORS are known violations retained deliberately for demonstration scope, and are the first items in the production hardening list (Section 7.1.3).
WCAG 2.1 [21]	The interface is the sole delivery channel and excludes users if inaccessible.	Semantic HTML controls, keyboard-operable question options, text-based mathematics rendered as accessible markup rather than images, and a bilingual interface reducing language exclusion. *Gap:* no formal audit against AA criteria was performed (Section 6.5.1).
Copyright in source textbooks and question banks	Content is derived from copyrighted printed material.	Source material is used as a private retrieval corpus, is not redistributed, and is excluded from the public repository's distributable artefacts; generated items are new text grounded in that corpus rather than reproductions of it.
IEEE/ACM codes of ethics [22, 23]	The system makes inferences that steer a student's study time during a high-stakes year.	Honest reporting of the system's unvalidated components, and refusal to claim score improvement without evidence, follow directly from these codes (Section 6.5.4).

Table 3.5: Evaluation of candidate solutions against criteria drawn from the requirements

Criterion (source)	A: IRT	B: DKT	C: BKT+DAG
Produces an actionable per-skill diagnosis (FR4, and the core problem statement)	No — one ability scalar cannot say *which* skill is missing	No — hidden vector is not interpretable per skill	**Yes**
Can express prerequisite structure (FR5, FR6)	No	Only implicitly, if data suffices	**Yes, explicitly**
Trainable with zero historical learner data (project constraint)	Needs calibration data per item	Needs large response datasets	**Yes — parameters set from pedagogical priors**
Interpretable to a teacher (teacher stakeholder)	Partly	No	**Yes**
Computational cost on commodity hardware (NFR6)	Low	High (training)	**Low**
Robust when the question bank is incomplete (project reality)	Poor — needs calibrated items	Poor — needs coverage	**Fair — fallback ladder plus propagation**
Implementation effort within one session	Medium	High	Medium

Conclusion. Alternative C was selected. The decisive criteria are the first and third rows. The project's entire premise is telling a student *which* foundation is missing, which A and B structurally cannot do: A compresses knowledge to one number, and B's state is a vector with no per-skill reading. Independently, both A and B require historical response data to calibrate, and the project began with none and would have none until it had users — a circular dependency that C avoids because its four parameters have defensible pedagogical defaults. B remains the most credible successor once real response data accumulates, and the engine was deliberately built so that the model can be replaced without touching its call sites (Section 4.2.1).

Chapter 4

System Design and Implementation

4.1 Functional Design and Sub-problem Partitioning

The problem decomposes into five subsystems with a deliberate dependency order. The knowledge ontology is upstream of everything: it defines the skills the engine reasons about, the topics the interface displays, and the specifications the content pipeline must generate against. The inference engine depends on the ontology but not on the content; the content pipeline depends on the ontology but not on the engine; the API composes engine and content; the interface depends only on the API. This ordering is what allowed the ontology to be rebuilt late in the project (Section 4.2.6) without rewriting the engine. Figure 4.1 follows a learner through the system before Figure 4.2 breaks it into components.

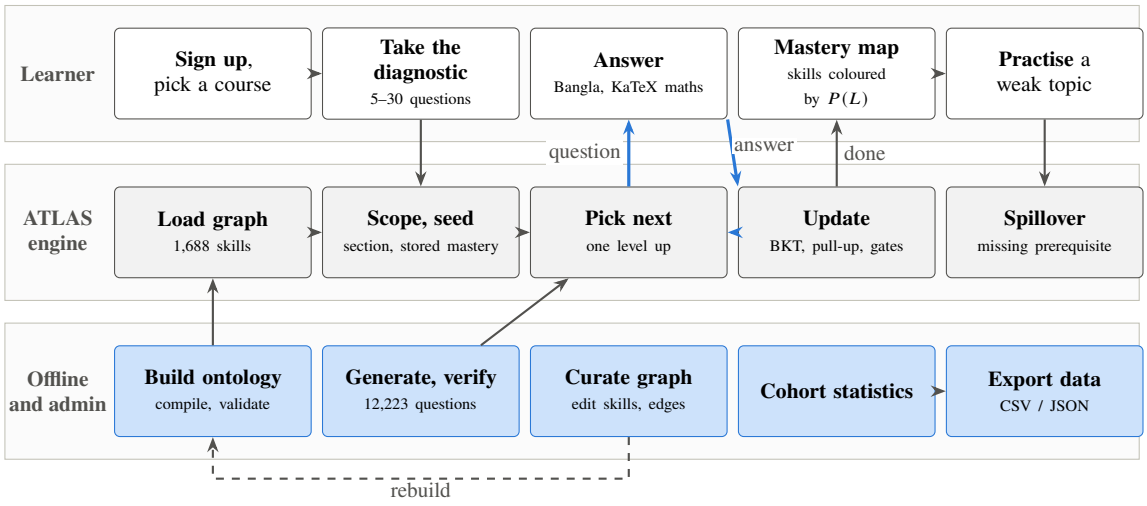


Figure 4.1: The learner's journey through ATLAS and what the engine and the offline and administrative side do at each step. The blue loop is the core of the system: each answer updates mastery and chooses the next question, until the section's diagnostic length is reached. Graph edits made by an administrator reach learners only after the ontology is rebuilt (dashed).

Where standards shaped the partition. Two boundaries in Figure 4.2 exist for reasons from Section 3.2 rather than for reasons of convenience. First, the privileged database key lives only in the API process: the client never holds a credential that can write to the database, which is why there is no direct client-to-database arrow even though the database vendor supports one.

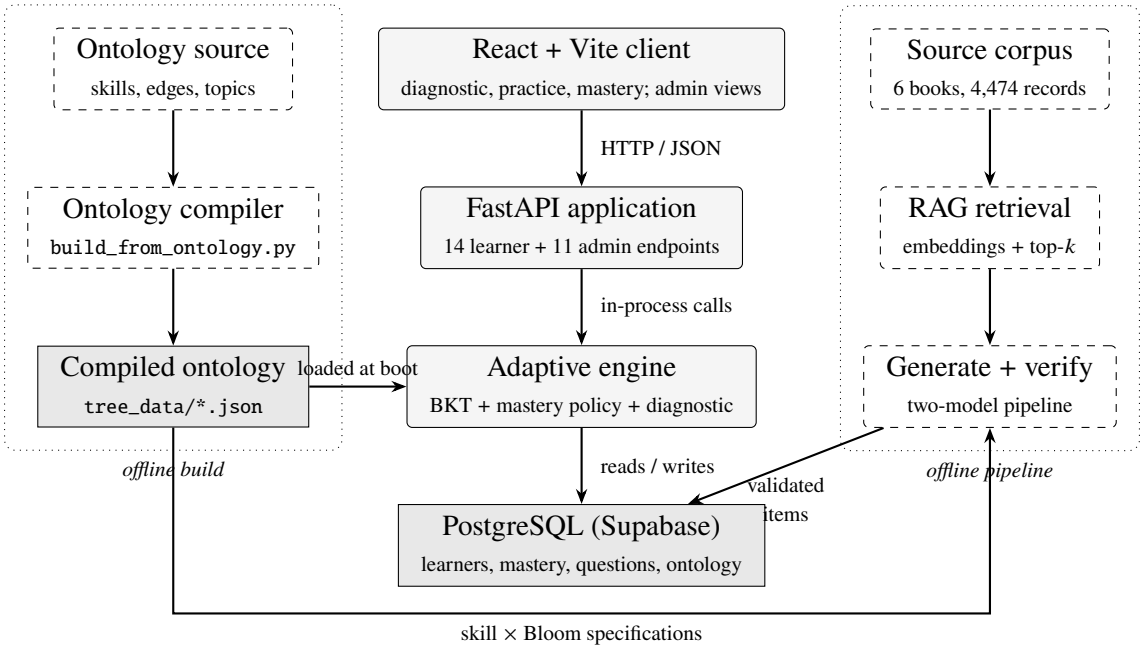


Figure 4.2: System architecture. Solid boxes are the online serving path; dashed boxes are offline processes that run before serving and write their artefacts into the compiled ontology and the question bank.

Second, the corpus sits strictly inside the offline pipeline and is never served: generated items reach the database, but the copyrighted source text never leaves the build environment.

Safety considerations. The system carries no physical-safety risk, but it does carry a decision-quality risk: a learner may reallocate study time during a high-stakes year on the strength of a wrong inference. The design response is conservatism in the gating policy — the threshold for declaring a prerequisite mastered is deliberately high (0.95), so the system prefers to keep a learner practising a foundation slightly too long over releasing them from it too early.

4.2 Design Refinement and Simulation

4.2.1 The knowledge-tracing model

Each skill's mastery is a latent binary variable; the engine maintains $P(L_n)$, the probability that the learner has learned the skill after n opportunities. Four parameters govern its evolution: the prior $P(L_0)$, the learning rate $P(T)$, the slip probability $P(S)$ and the guess probability $P(G)$ [2]; Figure 4.3 shows the model as a two-state process. Each response triggers a two-step update.

Table 4.1: Subsystem responsibilities

Subsystem	Location	Responsibility
Knowledge ontology	`Ontology/`, `Backend/tree_data/`	Defines skills, their Bloom levels, their topics, and the prerequisite edges between them. Compiled from source into the served JSON artefacts by a build script that enforces acyclicity and applies transitive reduction.
Inference engine	`Backend/bkt.py`, `bloom_taxonomy.py`, `mastery_updater.py`, `skill.py`, `skill_tree.py`	Maintains and updates per-skill mastery; owns the graph, its cycle check and its topological ordering.
Session logic	`Backend/diagnostic.py`, session state in `server.py`	Runs a diagnostic or practice session: which skill to test next, which Bloom level, which question, when to stop.
Serving API	`Backend/server.py`	Exposes catalogue, session and mastery endpoints; composes engine and database.
Content pipeline	`data-gen/`	Parses the corpus, builds a retrieval index, generates and verifies questions, loads them to the database.
Client	`Frontend/`	Renders the catalogue, question flow and mastery views; handles bilingual display and mathematical typesetting.
Administration	`Backend/admin_ontology.py`, `admin_stats.py`	Skill-graph curation against the ontology source, with cycle checks, and cohort statistics with CSV/JSON export. Access is gated by a username allow-list sent in a request header, which is not authentication (Section 5.3.3).

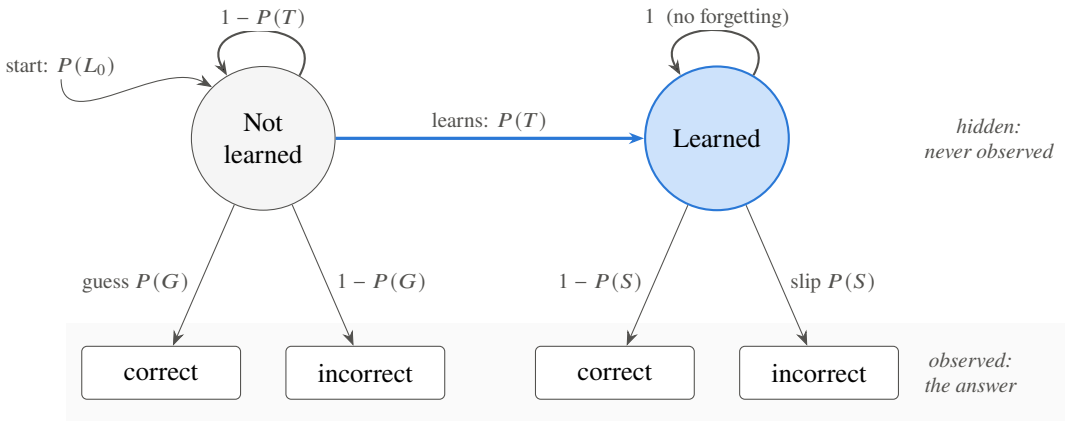


Figure 4.3: Bayesian Knowledge Tracing as a two-state hidden Markov model. The engine never sees which state a learner is in, only the answers, so it keeps $P(L)$, the probability of the *Learned* state, and revises it with Equations (4.1) to (4.3) after every response. Guess and slip are what make a single answer weak evidence: a right answer may be a guess, and a wrong one may be a slip.

Step 1, the Bayesian posterior given the observed response:

$$P(L_n \mid \text{correct}) = \frac{P(L_n)\,\big(1 - P(S)\big)}{P(L_n)\,\big(1 - P(S)\big) + \big(1 - P(L_n)\big)\,P(G)} \tag{4.1}$$

$$P(L_n \mid \text{incorrect}) = \frac{P(L_n)\,P(S)}{P(L_n)\,P(S) + \big(1 - P(L_n)\big)\,\big(1 - P(G)\big)} \tag{4.2}$$

Step 2, the learning transition — the chance that the opportunity itself taught the skill:

$$P(L_{n+1}) = P(L_n \mid \text{evidence}) + \big(1 - P(L_n \mid \text{evidence})\big)\,P(T) \tag{4.3}$$

Mastery is persisted as a percentage on $[0, 100]$ and related to the probability form by $P(L) = \text{mastery}/100$. The implementation is deliberately *stateless* — it takes a probability and an observation and returns a probability, holding no learner state — so that the entire model can be replaced by an individualised or neural variant [5, 10] without altering a single call site. That is the concrete design provision for the successor identified in Section 3.3.

4.2.2 Conditioning the noise parameters on cognitive level

The refinement that distinguishes this engine from textbook BKT is that $P(G)$ and $P(S)$ are not constants but functions of the question's Bloom level. A four-option recall item is guessable; an analysis item is not. Conversely, a learner who knows an analysis-level procedure has more opportunities to slip while executing it. Table 4.2 gives the values used.

Table 4.2: Bloom-conditioned guess and slip parameters, and the mastery band each level occupies

Bloom level	$P(G)$	$P(S)$	Mastery band
Remember	0.30	0.05	0.00 – 16.67
Understand	0.25	0.08	16.67 – 33.33
Apply	0.18	0.12	33.33 – 50.00
Analyze	0.12	0.18	50.00 – 66.67
Evaluate	0.08	0.22	66.67 – 83.33
Create	0.05	0.25	83.33 – 100.00

The 0–100 mastery scale is partitioned into six equal bands of $100/6 \approx 16.67$ points, one per level, so that a mastery value maps directly to a cognitive level. This is what makes the selection rule expressible: the next question is requested one band above the learner's current band, which is the zone-of-proximal-development rule [14] rendered arithmetically. Figure 4.4 draws the bands and the rule, and Figure 4.5 shows what the conditioning does to a single update.

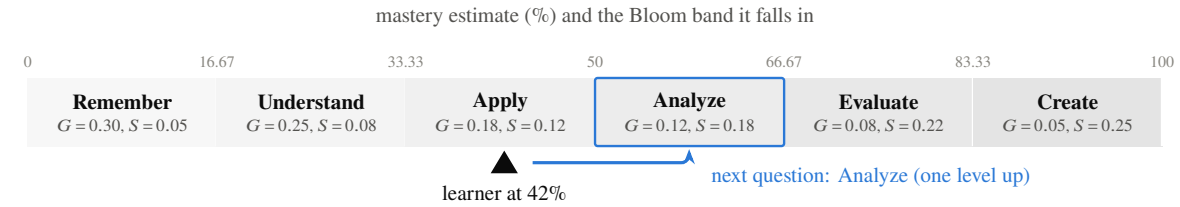


Figure 4.4: The 0–100 mastery scale is cut into six equal bands, one per Bloom level, each with its own guess and slip values (Table 4.2). The selection rule reads the band a learner is in and asks one level above it, capped at Create: a learner estimated at 42% sits in Apply, so the next question is an Analyze item.

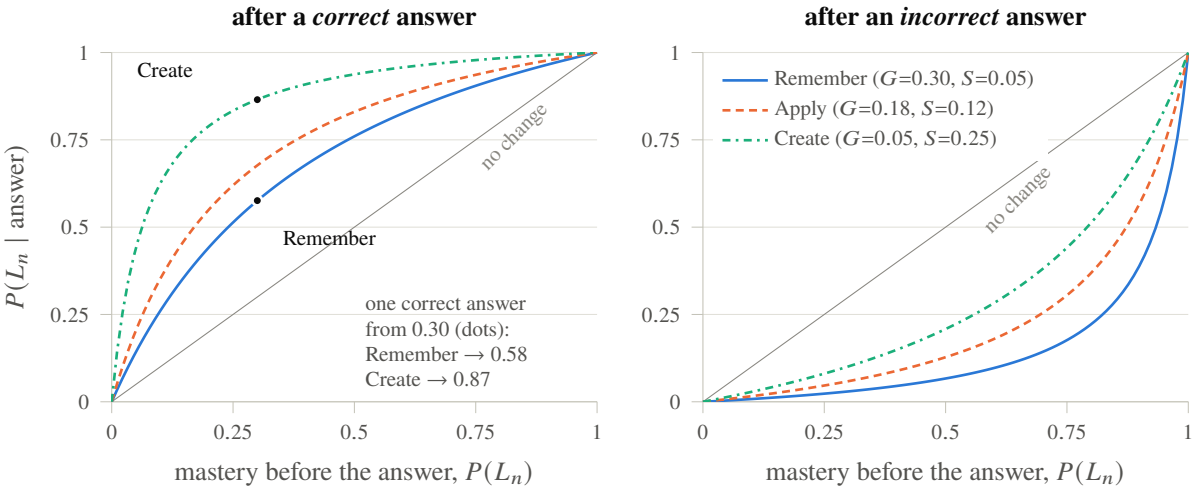


Figure 4.5: How far one answer moves the estimate depends on the question's Bloom level (Equations (4.1) and (4.2), parameters from Table 4.2; the learning transition is omitted to isolate the effect). A correct answer on a Create item is strong evidence because it is hard to guess; the same answer on a Remember item is weaker because nearly a third of non-knowers ($P(G) = 0.30$) would pick it anyway. Wrong answers show the mirror image: a miss on a Create item is partly forgiven as a slip.

4.2.3 The three-phase mastery update policy

Every response runs three phases in order (Figure 4.6).

Phase 1 — Bloom-conditioned BKT. Look up $P(G), P(S)$ for the question's Bloom level, compute the effective transition rate (Phase 3), and apply Equations (4.1) to (4.3) to the answered skill.

Phase 2 — Ancestor pull-up. If the response was correct, traverse the skill's ancestors breadth-first and raise each to at least the answered skill's new posterior:

$$P(a) \leftarrow \max\big(P(a),\, P(s)\big) \qquad \forall a \in \text{ancestors}(s) \tag{4.4}$$

The justification is evidential: correctly executing a skill is evidence that its prerequisites are held, because the skill is not executable without them. The asymmetry is deliberate — an *incorrect* response propagates nothing downward to prerequisites, because failure is ambiguous. It may indicate a missing prerequisite, but it may equally indicate a slip at the top level, and penalising

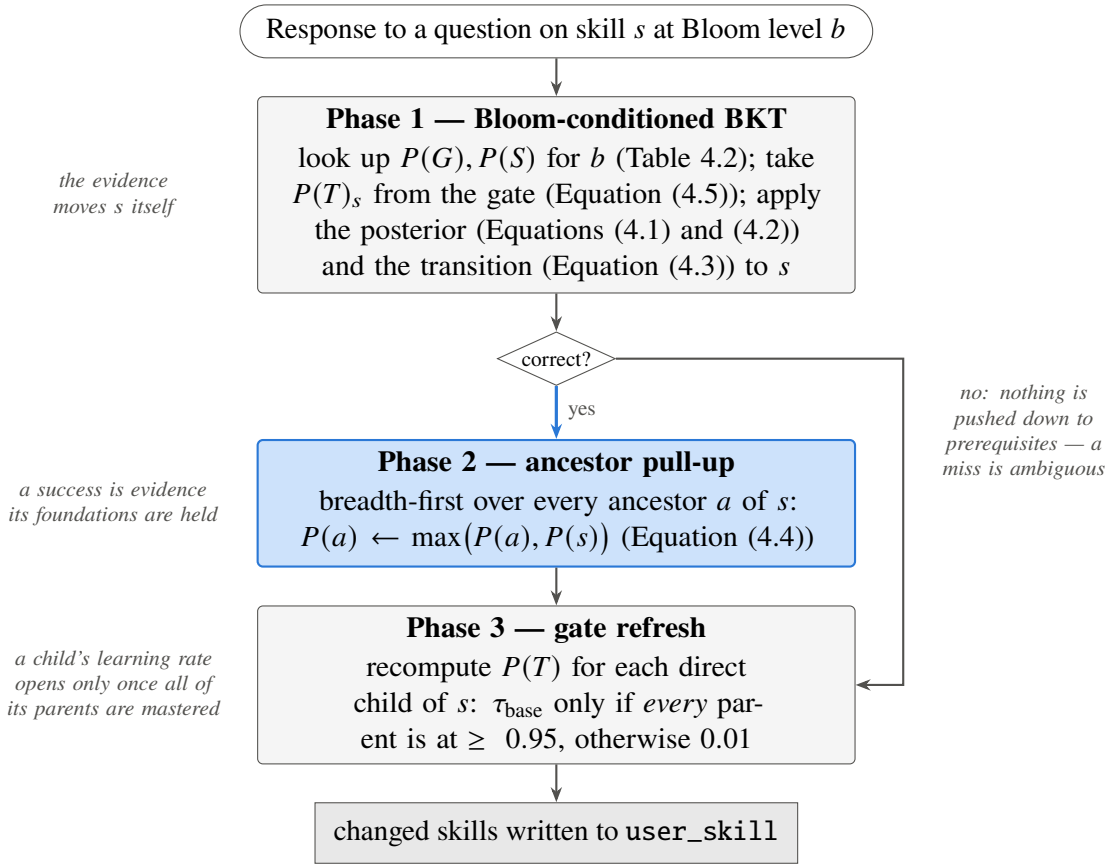


Figure 4.6: The three-phase update that every response triggers. Phase 1 is textbook BKT with Bloom-conditioned noise; Phase 2 lifts every ancestor to at least the answered skill after a success; Phase 3 keeps each child's learning rate in step with its parents. Only correct answers propagate, which is also why a later wrong answer can break the ordering Phase 2 set (Section 5.3.8).

every ancestor for one wrong answer would make the estimate unstable. Locating the responsible prerequisite is instead the job of the spillover rules, which use option-level evidence rather than a blanket penalty. The ordering pull-up establishes therefore holds at the moment of each correct answer, not for ever: a later wrong answer lowers the answered skill alone, and can leave it below a descendant credited earlier (Section 5.3.8). Figure 4.7 runs the policy on the skill fragment of Figure 4.12.

Phase 3 — Conjunctive transition gating. A skill's learning rate is not constant. For a skill s with direct prerequisites parents(s):

$$P(T)_s = \begin{cases} \tau_{\text{base}} & \text{if } \min_{p \in \text{parents}(s)} P(L_p) \geq \theta \\ \tau_{\text{locked}} & \text{otherwise} \end{cases} \tag{4.5}$$

with $\theta = 0.95$, $\tau_{\text{base}} = 0.10$ in normal practice, 0.20 during a diagnostic (where the aim is to move the estimate quickly), and $\tau_{\text{locked}} = 0.01$. The gate is *conjunctive* — every prerequisite must clear the threshold, not merely the average or the best one. What the gate withholds is the learning-transition credit of Equation (4.3): a skill whose foundations are not yet mastered is

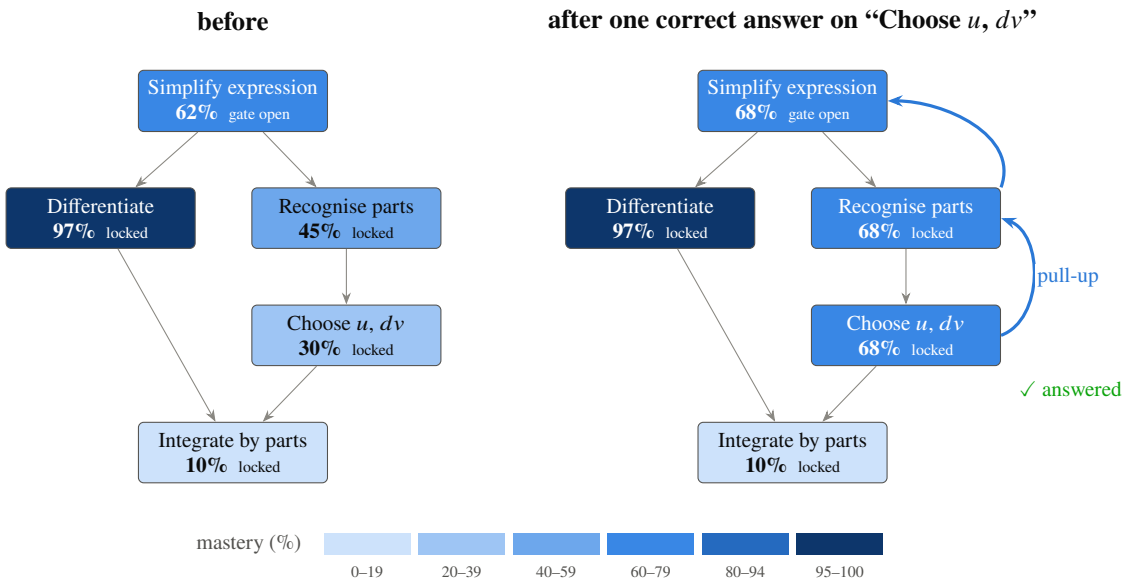


Figure 4.7: The policy on the skill fragment of Figure 4.12. One correct answer on "Choose u and dv" raises it from 30% to 68% (Phase 1; its gate is locked because "Recognise parts" is below 95%, so it learns at 0.01). Phase 2 then lifts both of its ancestors to the same 68%. "Integrate by parts" is untouched: pull-up never flows downward, and its gate stays locked until both of its parents reach 95%.

not assumed to be learning from practice. It does not cap the evidence itself — correct answers still raise the skill through Equation (4.1), and through Equation (4.4) they raise its foundations with it — so the ordering between a skill and its prerequisites is carried by pull-up, not by the gate. Figure 4.8 measures the difference on a single skill. The gate's correctness depends entirely on parents(s) being the *direct* prerequisites, which is why the ontology build applies transitive reduction (Section 4.2.6); a redundant shortcut edge would let a skill unlock while its true intermediate prerequisite was still unmastered.

4.2.4 Adaptive question selection

A diagnostic of tractable length cannot test 1,688 skills — a section's worth is already far more than a learner will sit through — so it must choose skills whose outcome is maximally informative about the rest of the graph. Only the skills of the section under test are candidates; the rest of the graph takes part through propagation alone. The engine scores each untested candidate by a balanced-volume heuristic:

$$\text{score}(s) = \frac{u_{\uparrow}(s) + u_{\downarrow}(s) + 1}{1 + |u_{\uparrow}(s) - u_{\downarrow}(s)|} \tag{4.6}$$

where $u_{\uparrow}$ and $u_{\downarrow}$ are the counts of untested ancestors and untested descendants. The numerator rewards skills that touch many unknowns; the denominator rewards *balance*. A skill with 20 unknown descendants and no unknown ancestors is a root whose failure tells us little about what lies above it; a skill sitting between two large unknown regions bisects the graph, and its outcome constrains both directions. This is the knowledge-space insight [11] applied as a greedy heuristic;

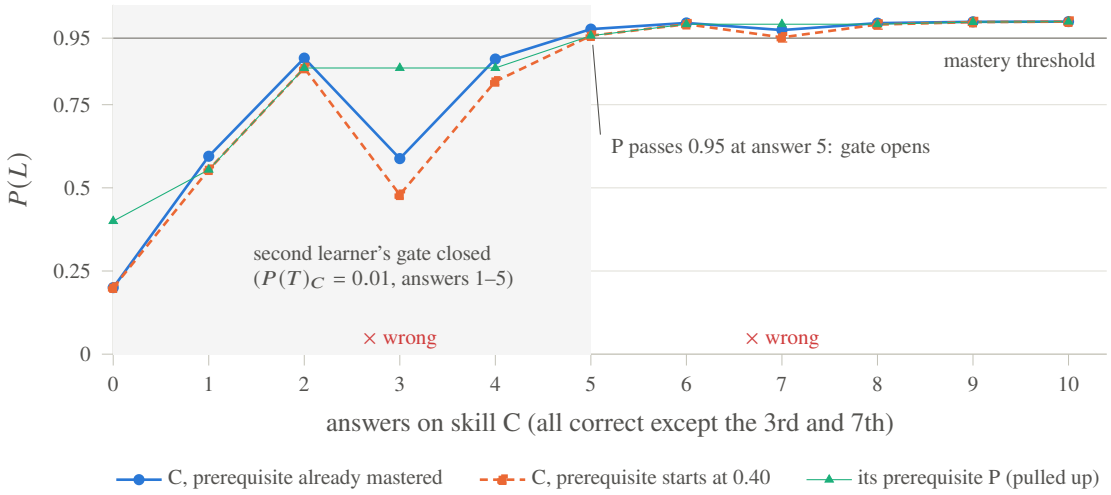


Figure 4.8: The conjunctive gate in practice, computed by the engine. Two learners give the same answers on a skill C whose one prerequisite P is either already mastered (gate open, $P(T)_C = 0.10$) or starts at 0.40 (gate closed, $P(T)_C = 0.01$). The Bayesian evidence dominates: the closed-gate learner trails by at most 0.11. Each correct answer on C also pulls P up to C's value, so P crosses 0.95 at the fifth answer and the gate opens. The gate therefore withholds only the unobserved learning credit; the ordering between a skill and its prerequisites comes from pull-up.

Figure 4.9 computes it for a real section.

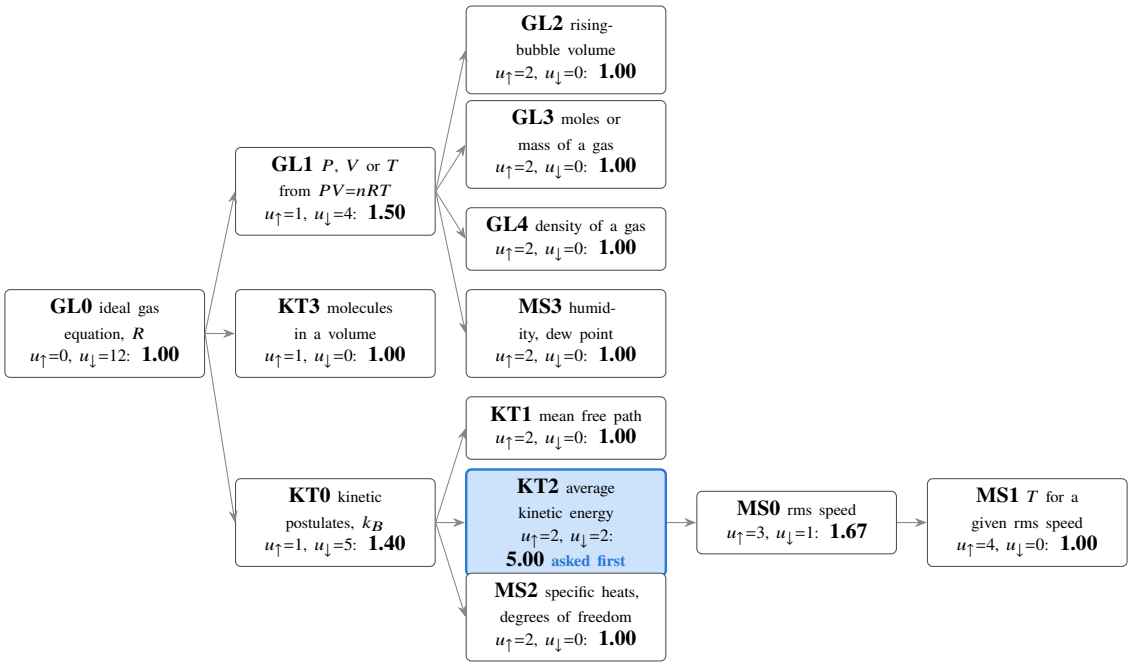


Figure 4.9: Choosing the first diagnostic question in the Ideal Gas and Kinetic Theory section (13 skills, 12 prerequisite edges; GL = gas laws, KT = kinetic theory, MS = molecular speeds). Each skill is scored by Equation (4.6) from its untested ancestors $u_\uparrow$ and descendants $u_\downarrow$. The root GL0 touches every skill but only from one side (1.00); KT2 sits between two unknown regions of equal size and scores 5.00, so its outcome says the most about both, and the live diagnostic asked it first.

Priors are initialised by depth: root skills start at 0.70 mastery and the deepest skills at 0.20, interpolated linearly between, encoding the assumption that foundational material is more likely

already held. The depth prior applies only where the learner has no stored mastery: a skill already recorded in `user_skill` starts the session at its stored value, so a diagnostic builds on earlier practice instead of overwriting it. A diagnostic asks one question per section skill, capped at 30 and never fewer than five.

Having chosen a skill and a target Bloom level, the engine searches for an item through a fallback ladder, because the ideal item frequently does not exist in a partially generated bank: (i) same skill, nearby Bloom level, matching topic; (ii) same skill, nearby Bloom level, any topic; (iii) same topic, graph-nearby skills ordered by distance, preferring the lowest-mastery candidate. "Nearby Bloom" means the offsets $[0, +1, -1, +2, -2, +3, -3]$ tried in that order. If no tier yields an item, the skill is marked tested and the session advances — the design decision that a missing question must never stall a session (NFR3). Figure 4.10 draws the ladder. Once an item is found, its options are shuffled and relabelled for this session, and the per-option explanations and distractor tags are withheld until the learner has answered; the answer is graded against the label the learner saw (Section 5.3.7).

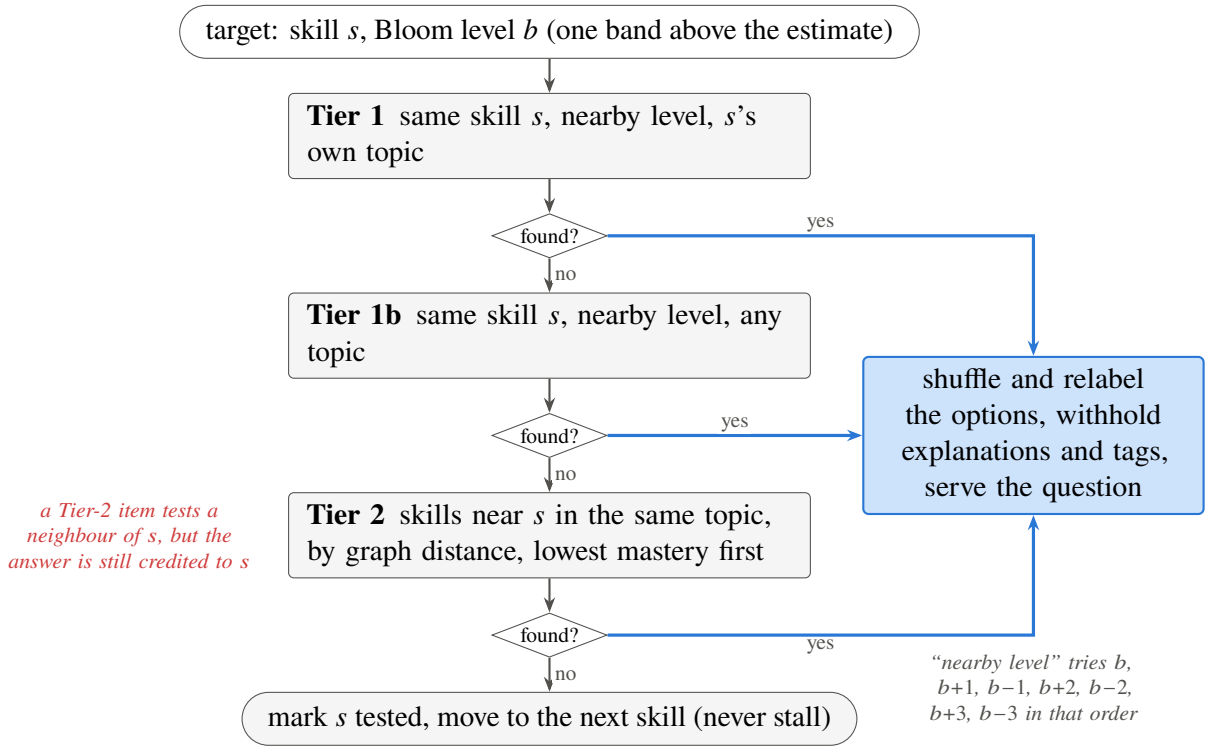


Figure 4.10: The fallback ladder that finds a question for the skill and level the diagnostic has chosen. Each tier widens the search; a skill with nothing to ask is skipped rather than stalling the session. Whatever tier supplies the item, the answer updates the *target* skill at the *target* level — the source of the attribution gap measured in Section 5.3.9.

4.2.5 Prerequisite spillover

During topic practice, two rules detect that failure is caused by something upstream rather than by the topic itself. Rule A fires when, among the last five answers, at least three wrong ones implicate the same missing prerequisite, evidence that comes from the `option_missing_prerequisites` table — each distractor is annotated with the skill whose absence would lead a learner to choose it. Rule A does not consult that prerequisite's own estimate. Rule B fires when accuracy over the last eight attempts falls below 40% *and* a prerequisite named by those wrong answers has mastery below 0.60; the weakest such prerequisite is chosen. Either rule injects two prerequisite-focused questions before returning to the topic traversal. The rules are evaluated only after a wrong answer whose distractor names a prerequisite, and only while no spillover is already running (Figure 4.11). Rule A is precise but needs distractor annotations; Rule B is a coarser safety net.

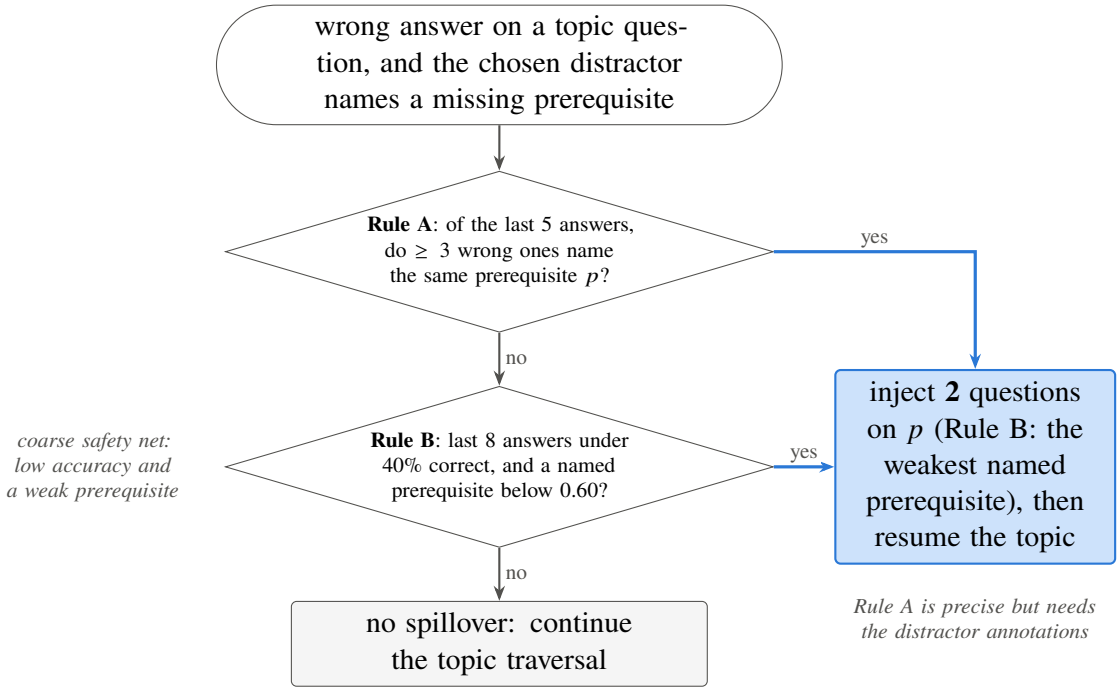


Figure 4.11: Prerequisite spillover during topic practice. Every distractor in the bank is annotated with the skill whose absence would lead a learner to choose it (`option_missing_prerequisites`); Rule A acts on that evidence directly, Rule B on accuracy and mastery. Spillover is considered only after a wrong answer whose distractor carries such a tag and only when no spillover is already running. Figure 5.3 shows Rule A firing in a live session.

4.2.6 The skill graph, and its rebuild

The first ontology was derived from a sample of the corpus and yielded 430 skills across 66 topics with 471 edges. In use it proved too coarse: a topic held on average fewer than four skills, so a failure inside a topic could not be localised further, and the conjunctive gate had too few

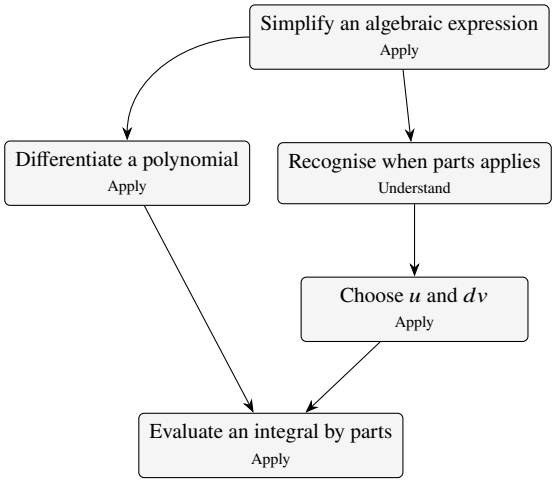


Figure 4.12: A fragment of the skill graph, illustrating the granularity target. "Integration by parts" is not one node but a chain: recognising applicability, choosing the decomposition, and executing it — each separately testable, and each with its own prerequisites, so that a failure can be localised to the step that actually broke.

edges to act on. The graph was therefore rebuilt from the complete corpus — all 4,474 parsed records of all six books — at a granularity where a node names one action, and the rebuild is now the catalogue the application serves. Figure 4.13 shows how it was built.

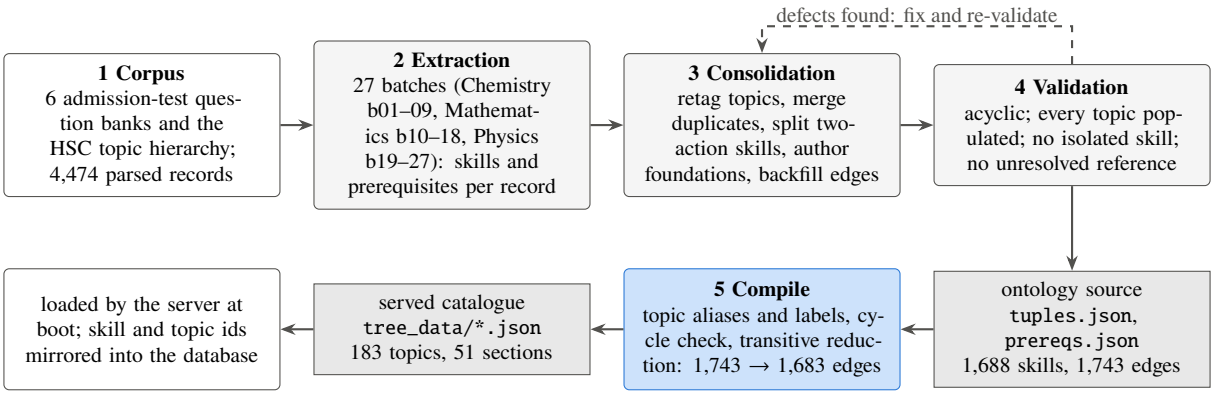


Figure 4.13: How the skill graph was built. Stages 1–4 produced the ontology source offline; stage 5 is a one-command, reproducible compile, so the served catalogue is never edited by hand. Transitive reduction removes the 60 edges already implied by a longer path, which the conjunctive gate (Equation (4.5)) requires.

Table 4.3 compares the two. The two edge counts given there are both real and differ for a reason worth stating: the ontology source asserts 1,743 prerequisite relations, and the build's transitive reduction removes the 60 that are implied by a longer path, leaving 1,683 direct edges in the compiled catalogue. It is the compiled figure that Equation (4.5) acts on.

Figure 4.14 shows the served graph's depth profile and how it splits into connected components.

The build is reproducible rather than hand-maintained: a compiler reads the ontology source, applies an editorial layer (topic aliases and display labels), checks the graph for cycles, applies transitive reduction, and emits the JSON artefacts the server loads. Transitive reduction is not

Table 4.3: Legacy ontology versus full-corpus rebuild

Property	Legacy (superseded)	Rebuild (served)
Skills	430	1,688
Prerequisite edges (source)	471	1,743
Prerequisite edges (compiled)	471	1,683
Topics covered	66	183 of 183
Corpus coverage	sample	all 4,474 records
Maximum graph depth	7	9
Isolated skills	—	0
Unresolved prerequisite references	—	0
Connected components (largest)	—	87 (482)

Table 4.4: Rebuilt ontology by subject

Subject	Skills	Edges	Edges per skill	Topics
Chemistry	886	868	0.97	59
Mathematics	446	499	1.10	52
Physics	356	376	1.06	72
Total	1,688	1,743	1.03	183

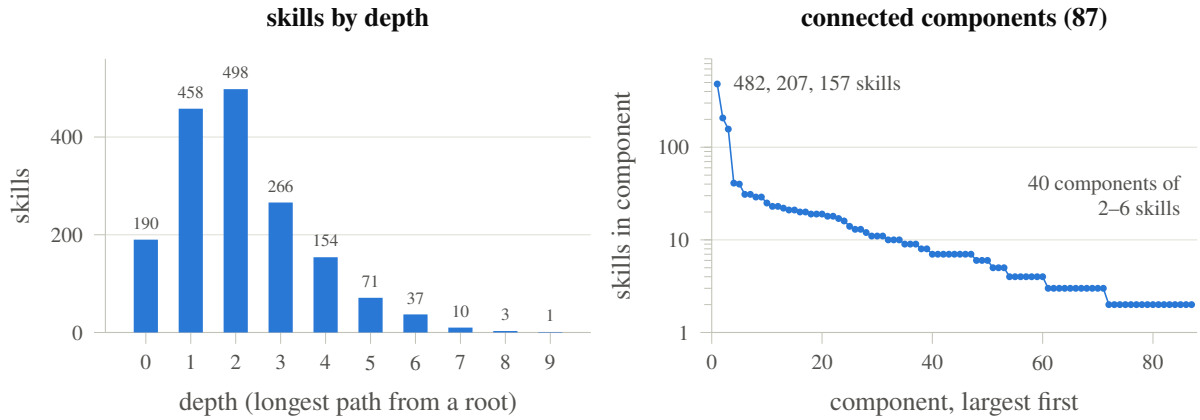


Figure 4.14: Shape of the served skill graph (1,688 skills, 1,683 direct edges). *Left:* most skills sit one to three prerequisites deep; the deepest chain has nine. *Right:* the graph is not one piece. Three large components hold 846 skills between them, and the long tail of small fragments is where a missing edge most often goes unnoticed. Of the 40 components of 2–6 skills, 32 are the near-orphan fragments of Table 5.1 still awaiting a domain expert (Section 5.1); the other 8 share their topic with no lower-tier skill outside themselves, so there is nothing within the topic to attach them to.

cosmetic — as Equation (4.5) shows, a shortcut edge would let a skill unlock while its true intermediate prerequisite was unmastered.

4.2.7 Data model

Table 4.5: Database schema

Table	Contents and constraints
`users`	Learner identity: surrogate UUID, unique username.
`user_skill`	Per-learner, per-skill mastery on $[0, 100]$; composite primary key; cascades on learner or skill deletion.
`skills`	Skill identifier and description; description is NOT NULL. Holds the 1,688 served skills and the legacy identifiers that older questions still reference (2,212 rows).
`questions`	Stem, Bloom level, and topic. `topic` is a foreign key to `ontology_topics`, so it stores a topic *code*, never a display label.
`question_options`	Four options per question, label constrained to A–D, correctness flag, and a per-option explanation.
`option_missing_prerequisites`	Maps a distractor to the skill whose absence explains choosing it — the evidence source for spillover Rule A.
`ontology_topics`, `ontology_skill_topics`, `ontology_skill_edges`	Topic codes, which `questions.topic` references, and a mirror of topic memberships and edges. Only the topic codes are current: the memberships and edges still describe the legacy 430-skill ontology and are not read at runtime (Section 5.3.13).

Figure 4.15 shows the same schema with its foreign keys and the row counts found in the end-to-end check.

Two deliberate omissions are worth stating. There is no `subject` column anywhere: subject is a property of the ontology, derived at read time and plumbed through the API, so that the database never holds a second copy of a fact the ontology owns. And the transition probability $P(T)_s$ is not persisted: it is recomputed from current parent mastery on every request, so it can never fall out of sync with the values it is derived from.

4.2.8 API surface

The learner-facing API is a flat, session-oriented REST surface of 14 endpoints, and a further 11 administrative endpoints sit under `/admin/`: two for statistics and nine for ontology curation (Table 4.6). The two session families are deliberately symmetric — `start`, `status`, `next`, `answer` — so the client can drive either flow with one component. Figure 4.16 follows one diagnostic through the API.

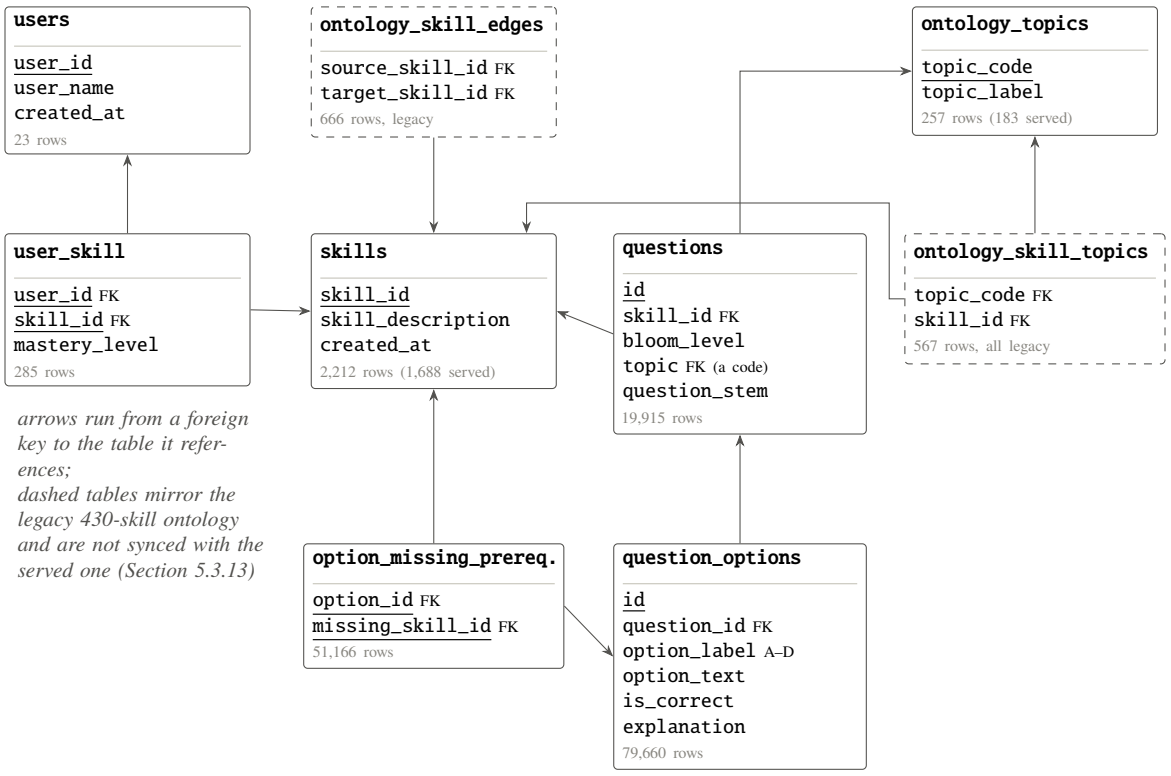


Figure 4.15: The database schema, with row counts from the end-to-end check. Learner state is a single table, `user_skill`; the transition probability is recomputed on every request rather than stored. Every question, option and distractor tag is tied to a skill by a foreign key, so the bank cannot reference a skill the database does not know.

4.2.9 Content generation pipeline

The pipeline turns the corpus into items aligned to the ontology. Source books are parsed into structured records; a multilingual embedding model [17] indexes them; for each (skill, Bloom level) specification the top-k relevant records ($k = 5$) are retrieved and supplied to a generator model as grounding [16]. Figure 4.17 shows the stages.

The design decision that matters most is the second stage. A generated item is not accepted on the generator's own word: a *different, cheaper* model (Gemini 3.1 Flash-Lite, against the generator's Gemini 3.5 Flash-Lite) independently checks that the item is on-topic for the claimed skill, and *blind re-solves* it — it is shown the question and options but not the marked answer — and the item is accepted only if its independent answer agrees with the marked one. Letting a model grade its own output biases towards false confidence; an independent judge with no sight of the expected answer is a real check. The cost asymmetry is deliberate: generation uses the stronger model because it is the harder task, verification uses the cheaper one because agreement checking is easier than authoring, and verification calls are batched across several specifications to reduce call count.

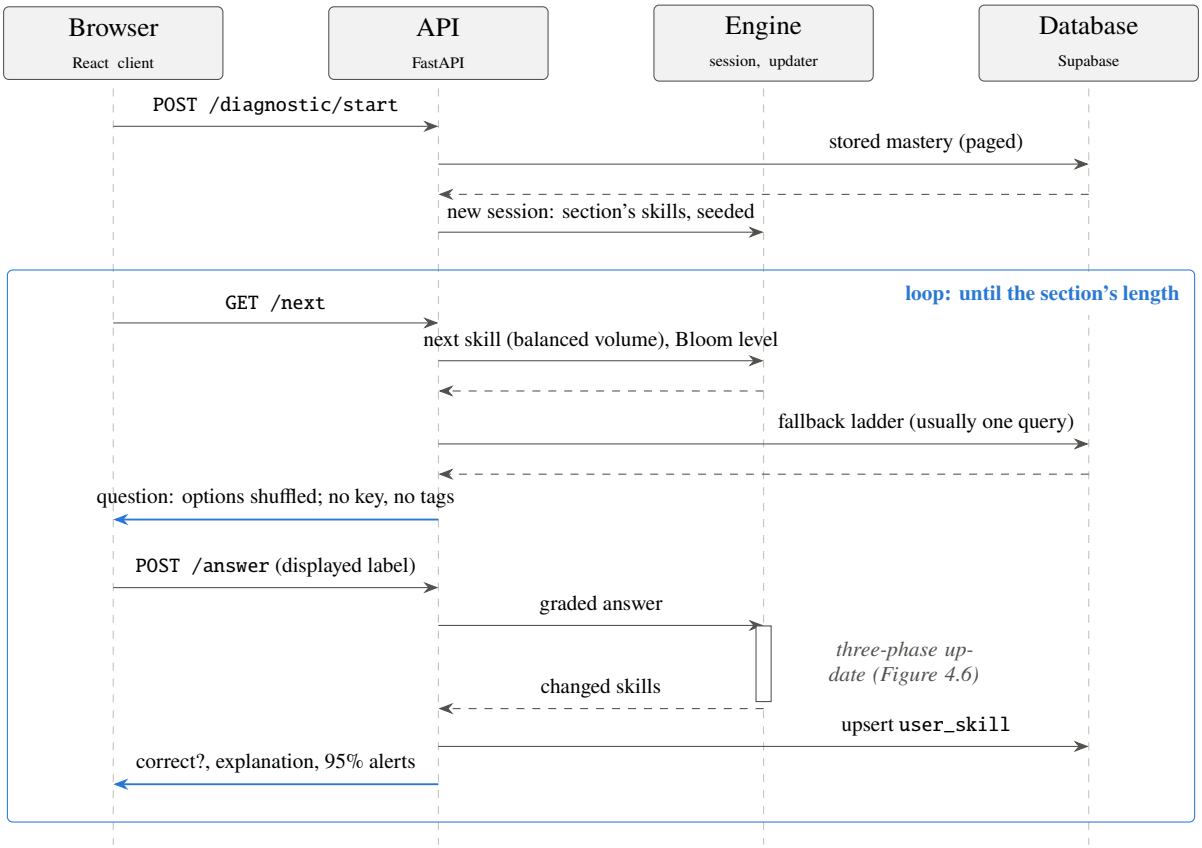
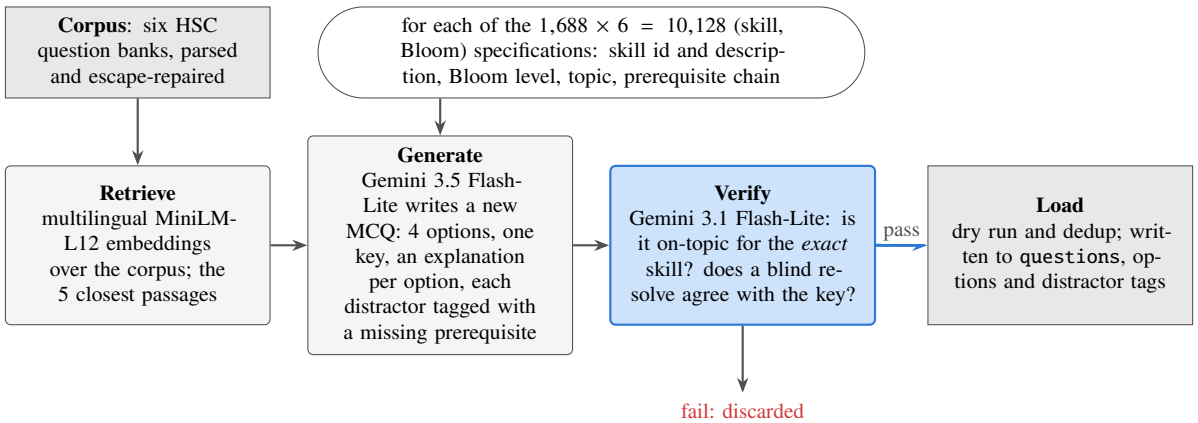


Figure 4.16: One diagnostic, request by request. Sessions live in the API process; only mastery is persisted. The answer key never leaves the server before the learner answers: options are shuffled when a question is first served, the shuffled copy is kept as the session's current question, and the answer is graded against the label the learner saw.



18 pooled API keys; rate limits tracked per (key, model); progress checkpointed so an interrupted run resumes. Outcome: 9,037 of the 10,128 specifications hold at least one verified item (Figure 4.18).

Figure 4.17: The question-generation pipeline. The decisive stage is verification by a different model that sees the question and options but not the marked answer: an item is kept only if that independent solve agrees with the key and the item tests the exact target skill.

Table 4.6: API endpoints

Endpoint	Purpose
`GET /catalog`	Courses, sections and topics from the compiled ontology
`GET /users/{name}/`, `POST /users/`	Look up and create a learner
`GET .../sections/{id}/state`	Diagnostic gate and lock state for a section
`GET .../sections/{id}/mastery`	Mastery graph and table for visualisation
`POST .../diagnostic/retake`	Reset section mastery and clear its sessions
`POST /diagnostic/start`	Begin a diagnostic session
`GET /diagnostic/{id}/status`, `/next`	Session progress; fetch next selected question
`POST /diagnostic/{id}/answer`	Submit a response; triggers the three-phase update
`POST /topic-practice/start`	Begin practice on one topic
`GET /topic-practice/{id}/status`, `/next`	Progress; next question, including injected spillover items
`POST /topic-practice/{id}/answer`	Submit a response; evaluates spillover rules
`GET /admin/stats/overview`, `/export`	Cohort statistics; full per-skill export as CSV or JSON
`/admin/ontology/...` (9 endpoints)	Read the ontology source (meta, topic graph, filtered graph); create, edit and delete skills and edges; rename topics

4.2.10 Design calculations

Generation space. With one item per (skill, Bloom level) pair, the legacy ontology implies $430 \times 6 = 2{,}580$ specifications; the rebuilt ontology implies $1{,}688 \times 6 = 10{,}128$. This is the dominant cost driver in the project (Section 6.1.2). The run against the rebuilt ontology has since been carried out: 9,037 of the 10,128 specifications (89%) hold at least one verified item, 12,223 questions in all, and 1,647 of the 1,688 skills can be asked about (Figure 4.18). Physics, at 74% of its specifications, is the thin subject.

Storage. A question with four options and explanations occupies on the order of a few kilobytes. A complete bank at rebuild scale is therefore of the order of tens of megabytes — immaterial against any database tier. Learner data is smaller still: one row per learner per skill, so a cohort of 10,000 learners against 1,688 skills is under 2×10^7 rows of two identifiers and a float, which is well within commodity capacity.

Inference cost. One response costs one constant-time Bayesian update plus an ancestor traversal bounded by graph depth (9) and a successor refresh over direct children. The engine is not the scaling bottleneck; database round trips during question search are, as the measurement in Figure 5.4 confirms.

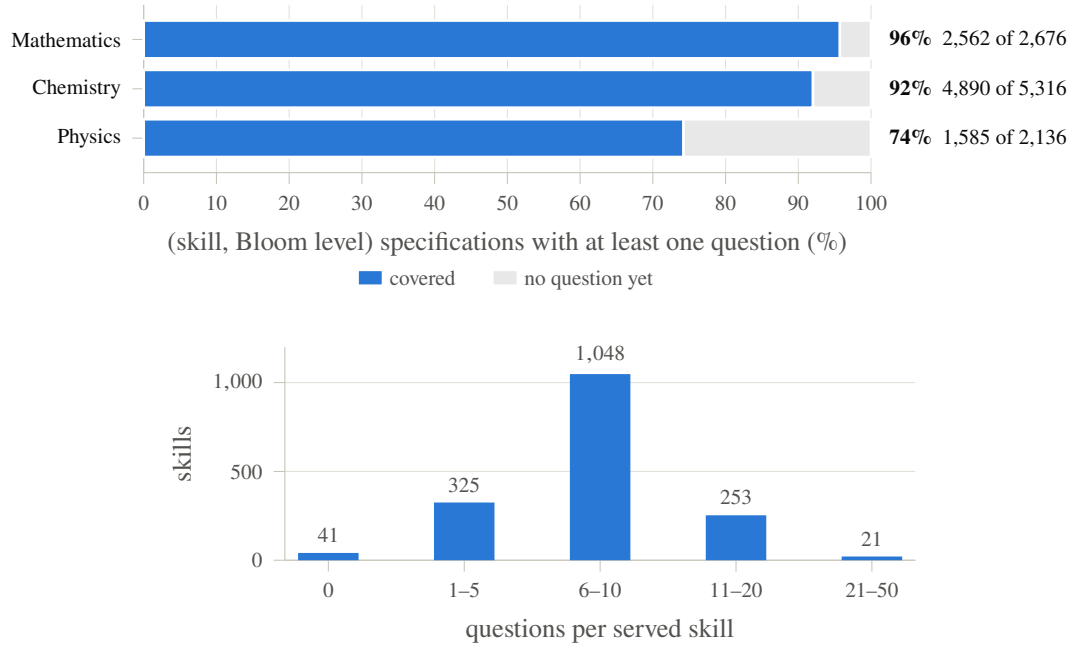


Figure 4.18: The question bank against the served ontology. *Top:* 9,037 of the $1{,}688 \times 6 = 10{,}128$ generation specifications (89%) hold at least one verified question; Physics is the thin subject. *Bottom:* a typical skill has six to ten questions spread over its Bloom levels, and 41 skills (31 of them in Physics) have none, which is what shortens some diagnostics (Section 5.3.10). A further 7,692 questions in the database are keyed to 487 legacy skill identifiers and are unreachable by the engine.

4.3 Application of Tools

Table 4.7: Tool selection, justification, and relevant limitations

Tool	Why selected	Relevant limitations
Python 3	The engine is mathematical and graph-oriented; Python keeps Equation (4.1)–Equation (4.5) readable enough to audit against the policy document.	Slower than a compiled language — immaterial here, since the update is constant-time.
FastAPI + Uvicorn	Typed request/response models via Pydantic catch malformed payloads at the boundary; automatic OpenAPI documentation made the API self-describing for the frontend developers.	The asynchronous model gives no benefit as used, since the engine is synchronous and CPU-light.

continued on next page

Table 4.7 — continued from previous page

Tool	Why selected	Relevant limitations
Supabase (PostgreSQL)	A managed relational store with a usable free tier. Relational integrity is not optional here — foreign keys are what stop a question referencing a skill that no longer exists.	The free tier pauses idle projects; the client library adds a network round trip where raw SQL would allow one query.
React 18 + Vite	Component model suits a question-at-a-time flow with shared state; Vite's fast rebuilds mattered on laptop hardware.	Single-page delivery means a large first-load bundle, worsened by mathematical typesetting fonts.
KaTeX	Renders LaTeX mathematics synchronously and quickly, and outputs accessible markup rather than images [21].	Supports a subset of LaTeX; unsupported macros in generated content must be caught before serving.
Tailwind CSS	Utility classes kept styling consistent across developers without a design system.	Verbose markup; class strings are hard to review in diffs.
Gemini API	After local generation proved infeasible on CPU (Section 2.4), a hosted model supplied the throughput; free-tier keys could be pooled to raise it further.	Rate limits shape the pipeline's architecture; a hosted model is a dependency the project does not control, and model updates can silently change output quality.
Sentence-transformers	Multilingual embeddings are required for a Bangla corpus; English-only models retrieve poorly here [17].	A large download and CPU-bound indexing; embedding quality on Bangla mathematical text is materially worse than on English prose.
Git and GitHub	Version control and the integration point for five developers working on separable subsystems.	—

4.4 Implementation

Engine. The model was implemented from the equations rather than from a library, which was the right call for auditability: the policy document and the code state the same constants, and a reviewer can check one against the other. The graph enforces its own invariant — adding a prerequisite edge provisionally inserts it, runs a three-colour depth-first search over the whole

graph, and rolls the edge back if it would close a cycle. The graph cannot enter an invalid state through the public interface.

Ontology. Generated artefacts are never hand-edited. The source is compiled by script, and the editorial decisions that cannot be derived from data — topic aliases, display labels, course and section layout — live in a single configuration file. Rebuilding is a one-command operation, which is what made a late full rebuild affordable.

Content pipeline. Two failures shaped the implementation more than any design intent. The first: source books are machine-extracted, and one corrupt record in the Chemistry corpus — a string containing a LaTeX macro repeated tens of thousands of times and never terminated — desynchronised the parser and destroyed 334 neighbouring records. The parser was rewritten to resynchronise by decoding one object at a time, so a bad record now costs only itself. The second: language models emit LaTeX with single backslashes, and a strict JSON parser silently consumes several of those escapes as control characters rather than raising an error, corrupting text invisibly. This affected a measured 13.4% of one early batch. The fix is an escape-repair pass applied before every parse, in both the ingest and generation paths. Both failures are documented in the repository so that a future contributor does not rediscover them.

Frontend. The mathematics rendering component is the non-obvious piece: it splits mixed Bangla-and-LaTeX strings on delimiters, renders the mathematical segments through KaTeX, renders the rest as text, and normalises newline sequences that arrive literally from generated content. That normalisation has a side effect found in the end-to-end check: it also splits a LaTeX command that begins with `\n`, such as `\neq` or `\nu` (Section 5.3.12).

Process. Work was carried out in a shared Git repository with subsystem-aligned branches merged into a main line — 119 commits between April and September 2026 from five contributors (Figure 6.2). In place of a formal CI service, the project relies on a credential-free test harness that any developer can run in seconds before merging (Section 5.2), plus a chain of handover documents that record the state and rationale of each subsystem at the point it changed hands. Those documents are themselves an engineering artefact: they exist because the project spanned several months and five people, and knowledge that lived only in one contributor's head was repeatedly found to be the project's real bottleneck.

4.5 Deployment

Target environment. The system deploys as two processes and one managed service: a Python ASGI application serving the API, a static bundle for the client, and the hosted PostgreSQL instance. A container definition is provided for the backend. Configuration is by environment variables — the database URL and service key — supplied through a file that is excluded from

version control; the client takes the API base URL at build time and defaults to the local backend.

Release process. The ontology is compiled and its report inspected; generated questions are validated and loaded with a dry run first; the backend and client are then started. The deliberate ordering reflects the dependency chain in Figure 4.2: content cannot be loaded against an ontology that has not been compiled.

Operation and maintenance. The prototype is operated by the project team. Monitoring is limited to structured logging of question-search behaviour — each search prints which tier matched and how long the database query took, which is what made the topic-code defect in Section 5.3 visible. Backup is delegated to the managed database service.

Resources consumed. One small application process (well under a gigabyte of memory, since the loaded graph is a few thousand nodes), one static file bundle, and a database holding tens of megabytes. Offline, the pipeline consumes CPU for embedding the corpus and external API calls for generation — the only materially resource-hungry part of the system, and the subject of the environmental assessment in Section 6.4.

Chapter 5

Investigation and Evaluation

5.1 Experimental Investigation

This is a development-focused project, so the investigations reported here are those that validated — or refuted — specific design decisions. Four are presented. Each states the question it was asked to settle, the method, and the conclusion actually drawn, including the two cases where the investigation overturned the team's initial position.

5.1.1 Investigation 1: is local model inference viable for content generation?

Question. Can the question bank be generated on the team's own hardware using locally hosted open-weight models, avoiding any external API dependency and any cost?

Method. A generation pipeline was implemented against locally served quantised models (7B and 3B parameter classes) on CPU-only laptops, and measured on real specifications drawn from the ontology, with prompt sizes of roughly 4–6 k tokens and completions of roughly 3 k tokens per call.

Findings. Two independent failures. On throughput, the measured per-call latency extrapolated to multiple weeks of continuous computation for a single full pass over the 2,580-specification space then in scope — longer than the remaining project schedule, on machines also needed for development. On correctness, a subtler defect appeared: models emit LaTeX commands such as `\tan` and `\frac` with single backslashes, and a strict JSON parser does not reject these — for the subset of letters that happen to be legal JSON escapes it silently consumes the backslash and substitutes a control character. The corruption therefore produced no error at all, and was found only by reading output.

Conclusion. The local approach was abandoned on throughput grounds, and a hosted API adopted with key pooling to raise the rate ceiling. The escape defect was not specific to local models — it recurred with the hosted model at a measured 13.4% of one early production batch — so an escape-repair pass was made unconditional in both the ingest and generation paths. This is the clearest case in the project of an investigation changing the architecture: the design in

Section 4.2.9 exists because this one failed.

5.1.2 Investigation 2: is the ontology's granularity sufficient to support the gating policy?

Question. The conjunctive gate in Equation (4.5) acts on a skill's direct prerequisites. Does the first ontology supply enough structure for that gate to do anything?

Method. Structural analysis of the 430-skill graph: skills per topic, syllabus coverage, and the fraction of skills with no prerequisite edge at all.

Findings. Two deficits, both fatal to the gate. The graph averaged fewer than four skills per topic, and a topic holding three skills cannot localise a failure *within* itself — which is the system's entire purpose. And a substantial fraction of skills had no incoming edge at all; a gate with no parents to test degenerates to the ungated case, so for those skills Equation (4.5) was doing nothing.

Note that the binding measure here is coverage and connectedness, not the raw edges-per-skill ratio. That ratio is similar before and after the rebuild (1.1 against 1.03), and deliberately so: the rebuild added roughly four times as many skills, so holding the ratio while eliminating isolated skills entirely means the edges are distributed across the graph rather than concentrated in a few dense topics. A higher ratio achieved by piling edges onto already-connected skills would not have fixed either deficit above.

Conclusion. The ontology, not the engine, was the binding constraint. This motivated the full-corpus rebuild (Section 4.2.6), which raised the graph to 1,688 skills and 1,743 edges with every one of the 183 syllabus topics populated and no isolated skills. Table 5.1 reports the rebuild's structural effect.

Table 5.1: Structural effect of the ontology rebuild and its editorial pass

Property	Before editorial pass	After
Syllabus topics holding no skill	54	0
Unresolved prerequisite references	46	0
Isolated skills (no edge at all)	6	0
Topics with no internal learning order	5	0
Near-orphan fragments (2–6 skills)	91	32
Connected components	187	87
Largest connected component	203	482
Maximum DAG depth	7	9

5.1.3 Investigation 3: can prerequisite edges be proposed mechanically?

Question. After the rebuild, 91 fragments of 2–6 skills remained disconnected from the main graph. Can the missing edges be proposed automatically rather than authored by hand?

Method. Two scoring functions were implemented and their proposals read individually against the subject matter. The first scored candidate parents by token overlap with the orphaned skill. The second scored by *topic hub-ness* — preferring the candidate at the lowest Bloom tier with the most dependents.

Findings. Token overlap produced roughly 40% usable proposals. Its failures were systematic rather than random: it proposed lateral links between peers that merely share vocabulary — zero-order kinetics "requiring" first-order kinetics, the cosine rule "requiring" the sine rule — and in one case proposed a pair in both directions, which would have closed a cycle. Hub-based scoring raised usable proposals to roughly two thirds. Of 83 proposals finally reviewed, 48 were applied (several with the direction reversed from what was proposed) and 35 were rejected outright, their fragments left deliberately disconnected rather than wrongly connected.

Conclusion. Mechanical proposal is a useful accelerator and an unacceptable authority. A wrong prerequisite edge is worse than a missing one, because Equation (4.4) propagates along it and will credit a learner for a skill they have never demonstrated. The 32 fragments that survive are recorded as requiring a domain expert rather than another heuristic.

5.1.4 Investigation 4: do automated quality heuristics over-flag?

Question. Two validators were written to find defects in the generated ontology: one flagging skills that fuse two actions, and one flagging "Bloom inversions" — edges whose prerequisite sits at a *higher* Bloom tier than the skill requiring it. Are their outputs trustworthy?

Method. Every flag from both checks was read against the underlying skill or edge.

Findings. The multi-action check flagged 173 skills, of which over half were false positives, and was replaced by a stricter criterion yielding 47 candidates — of which 20 were genuine splits and 27 were rejected on reading. The Bloom-inversion check flagged 147 edges; hand review found *3* genuinely reversed. The check's premise was wrong, not the data: Bloom tier measures cognitive demand, not teaching order, and a low-tier fact *about* an operation is routinely learned after the operation itself. For example, recalling that a determinant with two equal rows is zero (Remember) legitimately depends on being able to evaluate a determinant (Apply).

Conclusion. The inversion check was downgraded from error to advisory rather than deleted, with its reasoning recorded so the next contributor does not reinstate it. The transferable finding is that on this project mechanical heuristics over-flagged by roughly the same large factor twice,

and that the correct response is to calibrate a heuristic against hand review before treating its output as a defect list. Figure 5.1 puts the numbers of Investigations 3 and 4 side by side.

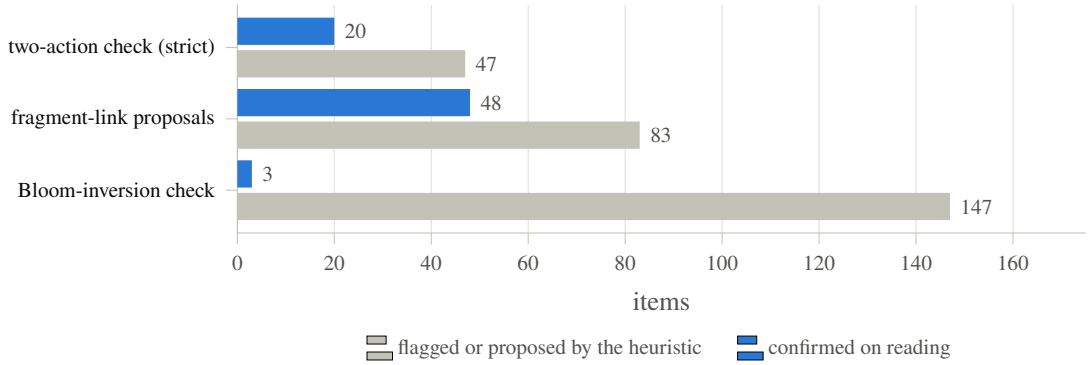


Figure 5.1: Mechanical heuristics against hand review. The Bloom-inversion check was right about 3 of its 147 flags; the strict two-action check about 20 of 47 (its first version flagged 173, over half of them wrongly); edge proposals for disconnected fragments were accepted 48 times in 83, several only after reversing their direction. Every flag was read before anything was changed.

5.2 Performance Evaluation

5.2.1 Setup and acceptance criteria

The engine is evaluated by an automated end-to-end harness that runs the real diagnostic, mastery-update, topic-practice and spillover code paths against an in-memory stand-in for the database, seeded with a synthetic learner and a question bank drawn from the compiled ontology. It requires no credentials and no network, which is what makes it runnable by any developer before a merge. Acceptance is binary per check: all checks must pass. Its stand-in database truncates every response at 1,000 rows, as the hosted one does, so that an unpaged read fails in the harness rather than only in production (Section 5.3.5).

A second, live check (Section 5.2.3) runs the same flows through the HTTP interface against the production question bank, in every section.

5.2.2 Results

The harness reports **29 checks passed, 0 failed**, reproduced on the current tree against the adopted 1,688-skill ontology. Table 5.2 lists what each check establishes.

One property of the harness should be read before its results are interpreted. It selects a section automatically and sizes the diagnostic to that section's skill count, so the absolute figures it prints depend on which section it picks — on the current tree that is a small one (eight skills). The

checks therefore establish that each mechanism *runs and holds its invariants*; they are not a measurement of how far evidence propagates in a large section, which would require a section with real depth and a populated question bank. The propagation guarantee is carried by the invariant checks, not by counting how many skills moved. Two synthetic learners are used: one answers every diagnostic question correctly, for the global ordering check; the other alternates right and wrong, so that topic practice has unmastered skills to work on and spillover can fire.

Table 5.2: End-to-end verification harness: checks and what each establishes

Area	Check	Establishes
Ontology	Graph builds (1,688 skills); catalogue topics all resolve (183); no legacy topic codes remain; all 1,688 skill ids load through the 1,000-row cap; no section resolves to zero skills	The served ontology is internally consistent, the catalogue cannot reference a topic that does not exist, and every section can be diagnosed
Question search	Tier-1 filter matches by topic *code*; filtering by display label matches nothing	Regression guard for the defect in Section 5.3
Diagnostic	Session starts, sized to the section (8 skills in the section exercised); selects only the section's skills, and every question served belongs to it; stored mastery seeds the session; question carries its subject; topic shown as label (`PHY1_ERRORS` → "Errors and Significant Figures"); every question answered	The adaptive loop stays inside its section, builds on earlier progress, runs to completion, and returns learner-presentable metadata (FR3, FR12)

continued on next page

Table 5.2 — continued from previous page

Area	Check	Establishes
Answer key	Options carry only label and text; shuffling keeps exactly one key and moves it, leaves the stored question untouched, and keeps the order of options that refer to one another	The client cannot read the answer from the payload or from its position (Section 5.3.7)
Mastery	Mastery unlocked after diagnostic; written to `user_skill`; section state reports subject	The update path persists per-skill mastery and releases the diagnostic gate (FR4, FR7)
Ordering	No ancestor below its descendant after the all-correct session; for the mixed learner, every ancestor at least as high as the answered skill right after each correct answer	Equation (4.4) behaves as specified. The global ordering after mixed answers is reported, not asserted (Section 5.3.8)
Mastery view	View unlocked; graph returns nodes and edges for the section; table rows carry subject	The visualisation contract holds (FR10)
Practice	Topic practice serves and accepts questions	Post-diagnostic flow works (FR8)
Spillover	Fires on repeated wrong answers, reporting its triggering rule (rule A, repeated missing prerequisite)	The prerequisite-detection policy triggers on the intended evidence (FR9)

The ontology is separately validated by a structural checker, which reports **zero errors**: the graph is acyclic, every one of the 183 topics holds at least one skill, no skill is isolated, and no prerequisite reference is unresolved.

5.2.3 Live end-to-end check

The harness proves that each mechanism runs on synthetic data; it cannot show how the system behaves on the real question bank. A second check therefore drives the actual FastAPI application through its HTTP interface, using an in-process client, against the production database. The question, option, skill and topic tables are read live. The learner tables are held in memory, seeded from the real rows, so the run cannot alter any learner's data; a write to any live table would have been blocked and reported, and none was attempted. The script creates a learner and, in every one of the 51 sections, takes the diagnostic with mixed answers, then a topic-practice session that begins with deliberate wrong answers. It also retakes a section, seeds a diagnostic from a real learner's stored mastery, and calls every administrative endpoint, running the ontology editor against scratch copies of its source files. Table 5.3 summarises the outcome.

Table 5.3: Live end-to-end check against the production question bank (51 sections, 4,862 requests, 14,066 assertions)

Flow	Result
Diagnostic	All 51 sections start and serve questions; 979 answers. Every question served belongs to its section, none repeats within a session, and grading of the shuffled label agrees with the bank's key in 979 of 979 cases, returning that option's explanation.
Topic practice	A session starts in every section; 510 answers, graded correctly in all 509 that could be matched (one question had two options with identical text). Spillover activated 84 times and served 130 prerequisite questions.
Retake and seeding	Retake clears the section's mastery and relocks it. A real learner's 64 stored values seed a new session unchanged, and starting a diagnostic writes nothing.
Administration	Statistics count every one of the 285 mastery rows and 23 learners; both exports are complete; every ontology-editor operation and error case behaves as specified on the scratch copies, and the real ontology files are unchanged afterwards.
Failures	4 of 77 checks (38 assertions): ten sections appear diagnosed before their own diagnostic (Section 5.3.11), and eight end the diagnostic short of its advertised length (Section 5.3.10).

Figures 5.2 and 5.3 follow one learner through one section with the same setup, the Ideal Gas and Kinetic Theory section, answering the first six diagnostic questions correctly and the rest wrongly; the questions served there are identical on every repetition. They show the mechanisms of Chapter 4 acting on real items: pull-up crediting a root skill before it is asked, spillover recognising that three failures share a prerequisite, and the session ending by itself once every topic skill passes the threshold.



Figure 5.2: The Ideal Gas section after a 13-question diagnostic against the live question bank, answering the first six correctly and the rest wrongly. Each node shows the depth-based prior and the estimate afterwards; ✓ and × mark the answers the engine credited to that skill. The correct answers on KT0 and GL1 lifted the root GL0 to 100% before it was asked (pull-up), while the wrong answers left their own skills near 40%. †MS2 has no question in the bank at the requested level, so the fallback ladder served an MS0 item in its place and the answer was credited to MS2 (Section 5.3.9).

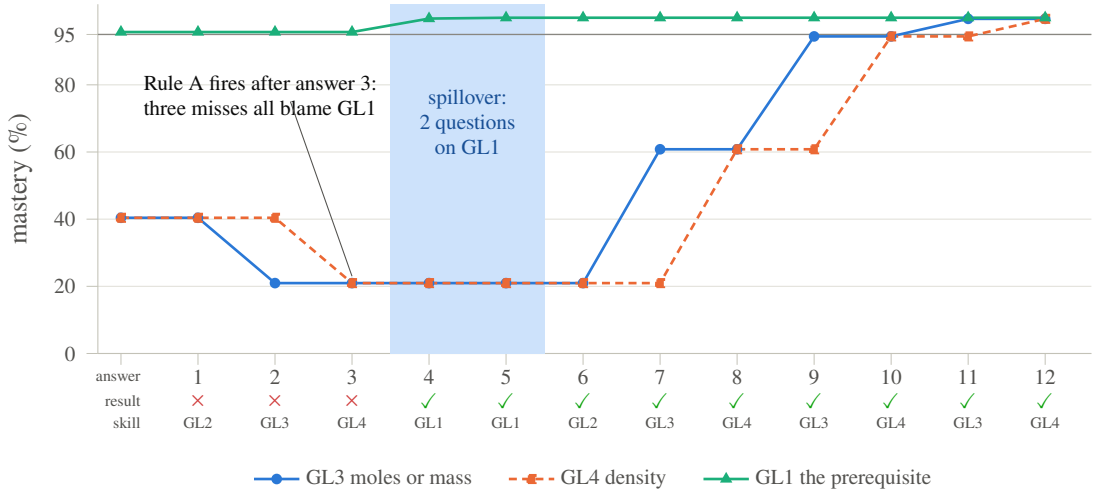


Figure 5.3: Topic practice on Ideal Gas Laws after the diagnostic of Figure 5.2, against the live bank. The first three answers are wrong, and every distractor chosen names GL1 as the missing prerequisite, so spillover Rule A fires and injects two GL1 questions (shaded) — although GL1 was already estimated at 96%, since Rule A does not consult the prerequisite's own estimate (Rule B does). Practice then returns to the topic and cycles through the skills still below the 95% threshold (grey line); each correct answer moves one skill by one Bloom-conditioned update. The session ends by itself after twelve answers, when all five topic skills pass the threshold (GL0 and GL2, already above it, are not plotted).

Responsiveness. The same run is the project's first measurement of NFR2 (Figure 5.4). Submitting an answer takes a median of 12 ms, because the update is arithmetic over the in-memory session. Choosing a new question takes a median of 180 ms but has a long tail — 1.1 s at the 90th percentile, 2.0 s at the 95th and 99 s in the worst case — and the tail sits in the Physics sections, where the bank is thinnest and the fallback ladder issues up to 100 queries for one selection. The figures come from a single client with no network between client and API, but with real round trips from the developer's machine to the hosted database, so they bound the latency a single learner sees from below and say nothing about concurrent load.

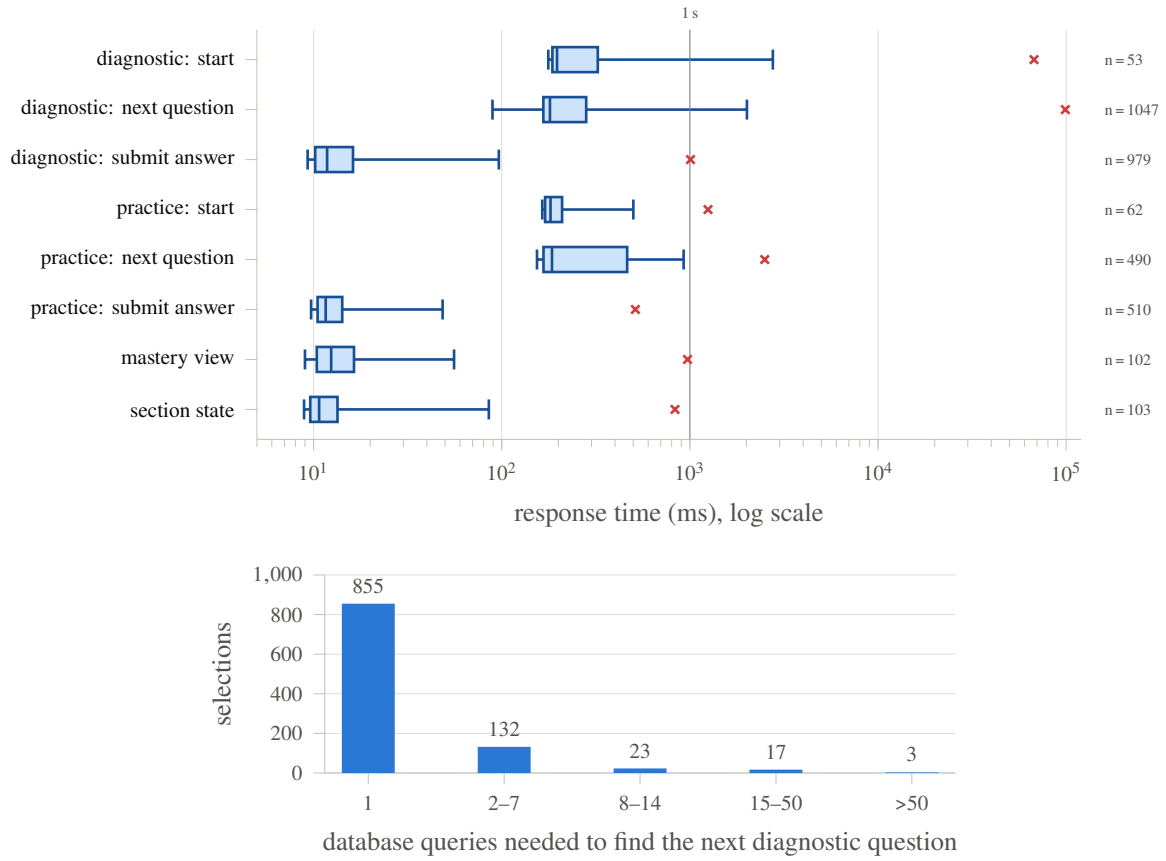


Figure 5.4: Responsiveness (NFR2), measured over 4,862 requests of the live end-to-end run. *Top:* boxes span the 25th–75th percentile with the median marked; whiskers reach the 5th and 95th; × is the slowest call. Answer submission is fast (median about 12 ms); choosing a new question has a median near 180 ms but a long tail. *Bottom:* the tail comes from the fallback ladder, counted by replaying the same code over a copy of the bank. 855 of 1,030 selections need a single query, but where the bank is thin the ladder issues one query per candidate (skill, level) pair, up to 100, and each is a round trip to the hosted database.

5.2.4 Evaluation against requirements

Table 5.4 traces every functional and non-functional requirement from Chapter 3 to the evidence that it is met, or to the deviation recorded in Section 5.3 where it is not. Entries marked as partly met, or met with a defect, point to the deviation that qualifies them; the evidence comes from the harness and from the live check of Section 5.2.3.

Table 5.4: Requirements traceability and status

Req.	Status	Evidence or deviation
FR1	Met, reduced	Account creation and sign-in work, but without a password (Section 5.3.3)
FR2	Met	Catalogue endpoint serves ontology-derived courses, sections and topics; all 51 sections resolve to skills
FR3	Met	Diagnostic runs to completion in the harness and in all 51 sections live; eight sections end short of their advertised length (Section 5.3.10)
FR4	Met	Bloom-conditioned update implemented per Table 4.2; exercised on every answered question; some answers are scored at the target rather than the served level (Section 5.3.9)
FR5	Met	Pull-up verified after every correct answer; the ordering it sets is not kept through later wrong answers (Section 5.3.8)
FR6	Met	Conjunctive gate implemented per Equation (4.5); its effect measured in Figure 4.8
FR7	Met, with a defect	Mastery view locked before, unlocked after the diagnostic; but ten of 51 sections unlock early (Section 5.3.11)
FR8	Met	Topic practice served and answered questions in the harness and in every section live
FR9	Met	Spillover fired and reported its triggering rule in the harness, and 84 times live
FR10	Met	Mastery graph (nodes and edges for the section) and table both returned
FR11	Met	Retake endpoint resets section mastery and clears sessions
FR12	Met, with defects	Mathematics renders through KaTeX, but 1,015 served questions (8.3%) contain content the renderer cannot display correctly (Section 5.3.12)
FR13	Largely met	9,037 of the 10,128 specifications (89%) hold a verified question; 41 skills have none (Figure 4.18)
FR14	Met	Interface strings available in English and Bangla
FR15	Met	Skill and edge creation, edits, deletion, cycle and duplicate rejection verified on scratch copies of the ontology source
FR16	Met	Statistics and both export formats verified against the live data
NFR1	**Partly met**	Acyclicity enforced at insertion and validated with zero errors; the ancestor ordering holds after correct answers but not through later wrong ones (Section 5.3.8)
NFR2	**Partly met**	Median 12 ms to submit an answer and 180 ms to select a question, but 2 s at the 95th percentile and up to 99 s in thin Physics sections (Figure 5.4)
NFR3	Met	Fallback ladder exercised live; a missing item marks the skill tested and advances; no session stalled
NFR4	Met	Ontology compiled from source by script; generated files are not hand-edited
NFR5	Met	Harness runs with no credentials and no network (29 checks)
NFR6	Met	Entire system developed and run on CPU-only laptops
NFR7	Met	Bangla content with LaTeX mathematics end to end
NFR8	Met	Credentials held in a git-ignored environment file, server-side only

5.2.5 Evaluation against objectives

O1, O2, O4, O5, O7 and O8 are met as evidenced above; O3 and O6 are qualified below.

O6 (question bank) is largely met, and what remains is worth quantifying rather than leaving as an adjective. The generation run against the adopted ontology has been carried out: 12,223 verified questions sit on the 1,688 served skills, and 9,037 of the $1{,}688 \times 6 = 10{,}128$ (skill, Bloom level) specifications — 89% — hold at least one (Figure 4.18). The shortfall is concentrated. Physics covers 74% of its specifications against 92% for Chemistry and 96% for Mathematics, and 41 skills, 31 of them in Physics, have no question at all. A further 7,692 questions in the database are keyed to 487 legacy skill identifiers and cannot be reached by the engine. Quality is now the larger gap than quantity: 8.3% of served questions carry a rendering defect (Section 5.3.12), and 38% of the distractor tags that spillover reads still name legacy skills.

O3 is met in implementation, with one qualification, and unvalidated in effect. Pull-up behaves as specified after every correct answer, but the ordering it establishes is not maintained when a later wrong answer lowers an ancestor (Figure 5.6). And whether the resulting estimates correspond to real learner knowledge is exactly the question the next subsection says was not answered.

5.2.6 Limitations of this evaluation

The evaluation above is structural and functional. It establishes that the system computes what it was designed to compute, and where it does not. It does **not** establish that the system helps anyone learn, and the report makes no such claim. Specifically:

- **No human participants.** There was no user study, no classroom trial, and no usability testing with members of the target cohort. Every claim about learner benefit in this report is an argument from design, not a measurement.
- **No fitted parameters.** $P(L_0)$, $P(T)$ and the Bloom-conditioned $P(G)$, $P(S)$ values are pedagogical priors, not estimates. The literature is explicit that fitted and individualised parameters outperform fixed ones [4, 5]; this system cannot do that until it has response data, which it cannot have before deployment.
- **No predictive validation.** The standard quantitative check on a knowledge-tracing model — does it predict the next response better than a baseline, measured by AUC on held-out data — was not performed, because there is no held-out learner data.
- **No load test.** Latency was measured for a single client (Figure 5.4); how the system behaves with many learners at once is unknown.
- **Ontology unreviewed by domain experts.** The rebuilt graph is structurally valid but its

subject-matter correctness has been verified only by the team. 32 disconnected fragments and a body of hand-authored prerequisite edges are explicitly awaiting review by a chemist and a physicist.

- **Question bank coverage is incomplete**: 89% of specifications overall, 74% in Physics. The consequences only the live check can surface, because the harness seeds synthetic items. Where the bank holds no item at the requested Bloom level, the nearby-level fallback ($[0, +1, -1, +2, -2, +3, -3]$) supplies whatever exists, so the cognitive progression is weaker in practice than in specification; and the answer is then scored with the guess and slip of the *requested* level. In the live check that was the case for 9.3% of diagnostic answers and 31% of practice answers, and 2.8% of diagnostic answers were credited to a different skill from the one the item tested (Section 5.3.9). Coverage narrows this gap; only a code change closes it.
- **The harness measures mechanism, not magnitude.** It sizes each run to the section it selects, so its printed figures vary with that choice and are evidence that each path executes and its invariants hold — not a measurement of how far evidence propagates across a large, well-connected section.

5.3 Deviations and Design Revisions

5.3.1 Topic filter matched no rows (found, analysed, fixed)

Deviation. The diagnostic's first two search tiers returned nothing, ever. Every question was silently arriving from the untargeted fallback, so topic-targeted and graph-nearby selection — two thirds of the selection design — were dead code in practice.

Cause. The `questions.topic` column is a foreign key to `ontology_topics` and therefore holds a topic *code* (`MAT_MATRIX`), while the search filtered it by the catalogue *display label* ("Matrices and Determinants"). The two can never be equal, so the filter matched no row. Nothing failed loudly: a query returning zero rows is a valid query, and the fallback dutifully supplied a question, so sessions looked healthy.

Revision. The filter was corrected to compare codes, and — more importantly — two checks were added to the harness: one asserting that a code-based filter matches, and one asserting that a label-based filter matches *nothing*. The second check is the one that matters; it fails if anyone reintroduces the confusion.

5.3.2 Session state is not persistent

Deviation. Diagnostic and practice sessions live in process memory, so a backend restart discards every in-flight session (NFR-adjacent, and a production defect).

Analysis. The session objects hold graph traversal state that was not designed for serialisation. Persisting them properly means either a session store or reconstructing session state from the durable mastery table on demand.

Decision. Not revised, deliberately. The deployment target for this project is a single demonstration process where the failure mode is a lost session rather than lost mastery — mastery itself *is* persisted. The correct fix is recorded as production hardening rather than disguised.

5.3.3 Authentication is username-only

Deviation. A learner signs in with a username and no password; the client holds the resulting identity in browser storage. This does not meet the access-control expectations of Section 3.2.

Analysis. The system stores no data that is sensitive in itself beyond performance history, and the prototype's users are the team. Authentication was therefore deprioritised against the ontology work, which was on the critical path for every other subsystem.

Decision. Not revised within the project. It is the first item of production hardening (Section 7.1.3), and is stated here rather than presented as a design choice, because it is not one — it is an accepted debt.

5.3.4 Ontology adoption, and the coverage debt it exposes

Deviation. For most of the project the application served the 430-skill legacy ontology while the validated 1,688-skill rebuild sat beside it, unadopted. Adoption has since been carried out: the compiled catalogue in `Backend/tree_data/` is now generated from the rebuild — 1,688 skills across all 183 syllabus topics — and the generation pipeline's ontology source has been repointed to the rebuild so that the two cannot drift apart.

Analysis. Adoption was deliberately held back rather than forgotten, because it is not merely a file swap. The existing question bank is keyed to legacy skill identifiers, so switching the catalogue does not carry the content across with it: a system that knows about 1,688 skills but holds questions for a fraction of them can diagnose more finely in principle while being able to ask about less in practice. Adopting therefore converts a *structural* deficit into a *coverage* deficit — the right trade, because coverage is bought by a generation run whereas structure had to be rebuilt by hand, but it is a debt and is recorded as one.

Decision. Adopt, then generate — the order finally taken. The generation run against the enlarged specification space has since been carried out and has repaid most of the debt: 89% of the specifications held a verified question as measured for the evaluation in Section 5.2.3 (Figure 4.18), rising to 94.1% (9,535 of 10,128 specifications, 12,343 questions, only 10 of 1,688 skills still with none) as the generation run continued past that measurement — the pipeline is designed to be resumable and was left running rather than stopped at the figure quoted for evaluation. Coverage figures elsewhere in this chapter are the evaluation snapshot and are not restated here. What remains is concentrated in Physics, and in 7,692 legacy-keyed questions that the engine cannot reach; Section 7.1.3 carries it.

The subsections that follow record the deviations found by the full end-to-end check of Section 5.2.3. The first three were fixed during that check; the rest are open and are stated with the measurement that exposed them.

5.3.5 Skill identifiers truncated at 1,000 rows (found, fixed)

Deviation. All 41 Physics and Mathematics sections resolved to no skills, so their diagnostics could not start. **Cause.** The hosted database interface truncates every response at 1,000 rows whatever the query asks for, and the server read its list of known skill identifiers — which filters every section — from the 2,212-row `skills` table in one unpaged request. The 1,000 rows it kept held all 886 Chemistry skills and none of the 802 Physics and Mathematics ones. **Revision.** The read now pages through the table. The harness's stand-in database truncates at 1,000 rows like the real one, and asserts that every identifier loads and that no section is empty.

5.3.6 Diagnostic ignored its section and stored mastery (found, fixed)

Deviation. The diagnostic chose skills from the whole 1,688-skill graph rather than from the section under test, so every section opened with the same Physics question; and it seeded every skill with the depth prior, overwriting mastery the learner had already earned. **Revision.** Candidates are now the section's skills only, stored mastery seeds the session, the prior applies only where nothing is stored, and the seeded values are not written back as if they were evidence. The harness checks all three; live, every one of the 979 diagnostic questions fell inside its section and 64 real stored values seeded a session unchanged.

5.3.7 Answer key readable by the client (found, fixed)

Deviation. Every question sent to the browser carried each option's explanation and missing-prerequisite tags. The correct option is the one with no missing prerequisite, so the answer could

be read from the network response. Independently, the bank stores the key as option A or B in 76% of questions (Figure 5.5). **Revision.** Questions now carry only each option's label and text; the chosen option's explanation is returned with the result. Options are shuffled when a question is first served, the shuffled copy is what the session grades, and the key's position is therefore uniform. The 23 questions whose options refer to one another by letter or position ("all of the above") keep their order.

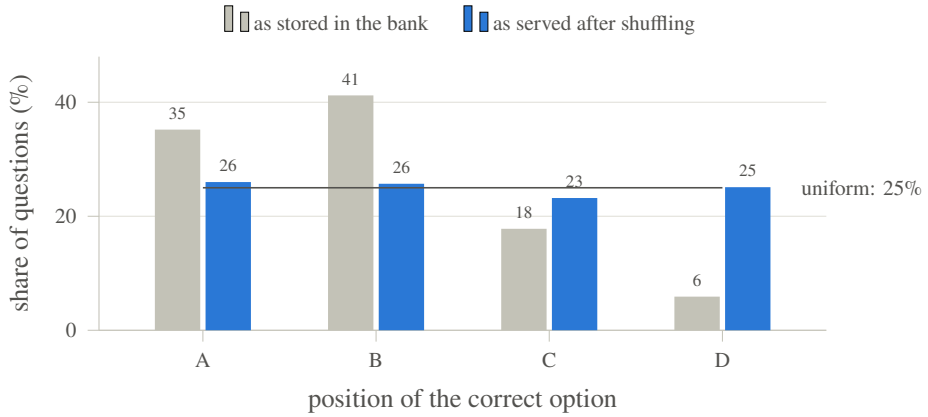


Figure 5.5: Where the correct answer sits. In the bank, the key is A or B in 76% of questions and D in only 6%, so a learner who always picked B would score 41% without reading a question. Options are now shuffled when a question is served (Section 5.3.7); across the 1,489 questions served in the live run the key's position is close to uniform.

5.3.8 Ancestor ordering not kept through wrong answers (found, open)

Deviation. NFR1 and the earlier harness stated that no skill ever sits below one of its descendants. That holds only while answers are correct. A wrong answer lowers the answered skill alone, so a skill whose descendant was credited earlier, or whose descendant is still at a higher prior, falls below it and stays there. The earlier harness answered every question correctly and never exercised the case. **Measurement.** Over all 51 sections, 4–8% of in-section edges end a mixed-answer diagnostic inverted (Figure 5.6). **Status.** Open. Restoring the order needs a policy decision — lower the descendant as well when an ancestor is failed, or cap the ancestor's fall at its highest descendant — and either changes what a wrong answer means, so it was not made unilaterally. The harness now asserts the property pull-up does guarantee and reports the inversions it finds.

5.3.9 Fallback items credited to the target skill (found, open)

Deviation. When the fallback ladder supplies an item from a neighbouring skill (Tier 2) or from a different Bloom level, the answer is still applied to the *target* skill with the *target* level's guess and slip. **Measurement.** In a diagnostic over all 51 sections, 27 of 979 answers (2.8%) were credited to a skill other than the one the item tested, and 91 (9.3%) were scored at a level other

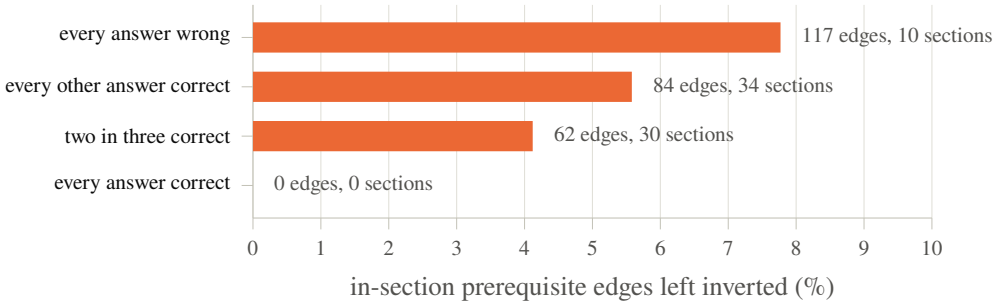


Figure 5.6: How often the ordering "no ancestor below its descendant" survives a diagnostic, over all 51 sections (1,505 in-section edges) for four answer patterns. With only correct answers it always holds, because pull-up runs after every success. A wrong answer lowers only the answered skill: if a descendant already sits higher — credited by an earlier correct answer, or simply still at its depth-based prior — the order is broken, and nothing restores it (Section 5.3.8).

than the item's; in topic practice, 114 of 371 answers (31%) were scored at the wrong level. Figure 5.2 shows one case. **Status.** Open; the remedy is to update the skill and level of the item actually served.

5.3.10 Diagnostic length overstated (found, open)

Deviation. A section's advertised diagnostic length is its skill count, capped at 30, but 41 skills have no question. Fourteen sections advertise more questions than they have skills to ask about; Tier 2 fills part of the gap, and in the live check eight sections ended early (Kinematics after 16 of 19), leaving a completion screen that reads as if questions were skipped. **Status.** Open; the remedy is to size the diagnostic by the skills that can be asked about.

5.3.11 Sections unlocked by another section's diagnostic (found, open)

Deviation. A section counts as diagnosed as soon as the learner has any stored mastery for any of its skills. Propagation crosses section boundaries — pull-up raises prerequisites that belong to neighbouring sections — so a diagnostic in one section can mark others as done. In the live check ten of the 51 sections appeared diagnosed before their own diagnostic, which skips the diagnostic prompt and opens practice on mostly untested skills. **Status.** Open; the remedy is to record completion explicitly for each learner and section. This and the ordering defect of Section 5.3.8 share one root cause: the system has no explicit, persisted notion of "this learner has diagnosed this section," so both a skill's rank relative to its descendants and a section's completion state are inferred from mastery values that pull-up can move for reasons unrelated to either — a single completion record per learner and section would close both.

5.3.12 Rendering defects in generated content (found, open)

Deviation. Replaying the client's own splitting logic and KaTeX over every stored stem, option and explanation shows 1,015 served questions (8.3%) that do not display correctly: 854 contain LaTeX outside the mathematics delimiters, which appears as raw source; 238 have unbalanced delimiters; 202 contain a command beginning with \n that the newline normalisation splits; and 111 contain LaTeX KaTeX cannot parse (a question may have more than one). **Status.** Open; the remedies are a repair pass at load time and a normalisation that leaves mathematics alone.

5.3.13 Ontology mirror in the database is stale (found, open)

Deviation. The database's copy of topic memberships and prerequisite edges still describes the legacy ontology: none of its 567 memberships and only 2 of its 666 edges match the served graph. **Analysis.** Nothing misbehaves today, because the engine loads the graph from the compiled catalogue and the foreign keys that matter — question to skill and to topic code — point at current rows. But any query that reads structure from the database gets the wrong graph. **Status.** Open; sync the two tables from the compiled catalogue, or drop them.

Chapter 6

Project Management, Teamwork, and Impact

6.1 Project Management and Finance

6.1.1 Planning and Risk Management

The project ran across one academic session with five contributors, all carrying a full course load. Work was partitioned along the subsystem boundaries of Figure 4.2, which was chosen partly for architectural reasons and partly because those boundaries are the ones that let several people work without blocking each other: the engine, the content pipeline and the client each depend on the ontology's *interface* but not on each other's internals.

Figure 6.1 shows the schedule. The version-control history (Figure 6.2) records 119 commits between April and September 2026 in three bursts — the engine, API and first interface in April; retrieval and the first per-subject ontologies in July; the generation pipeline, the ontology rebuild, the administrator tools and this report in September. No work reached the shared repository between the bursts, so the chart is the plan of record rather than a trace of the commit history.

Where actual progress differed from plan. Three deviations, each instructive.

1. **A late, unplanned ontology rebuild.** The plan assumed the knowledge structure was a one-time input. It turned out to be the project's critical path: the first ontology was too sparse for the gating policy to act on, and rebuilding it consumed most of September. The schedule absorbed this only because the compiler-based build made a rebuild a one-command operation.
2. **A discarded content-generation approach.** Weeks of work on local model inference were written off when measurement showed it could not finish in the remaining schedule (Investigation 1). Migrating to a hosted API recovered the capability but consumed schedule that had been allocated to coverage.
3. **Scope reduction on authentication.** Deferred deliberately once it became clear the ontology work would not otherwise finish.

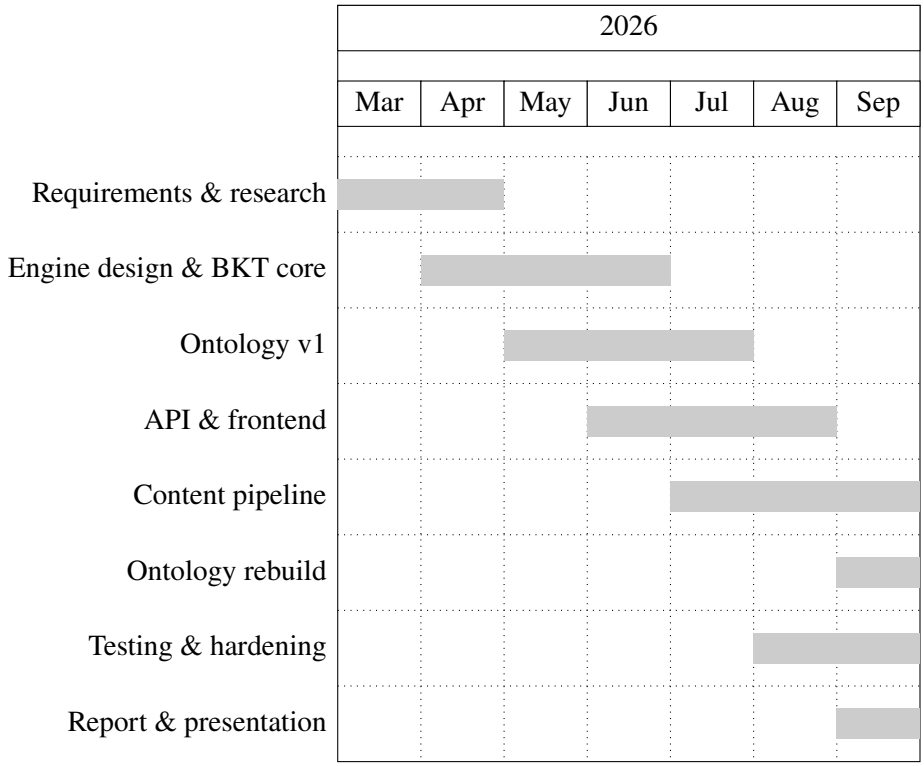


Figure 6.1: Project schedule. The ontology rebuild in September was not in the original plan; it was forced by Investigation 2 (Section 5.1).

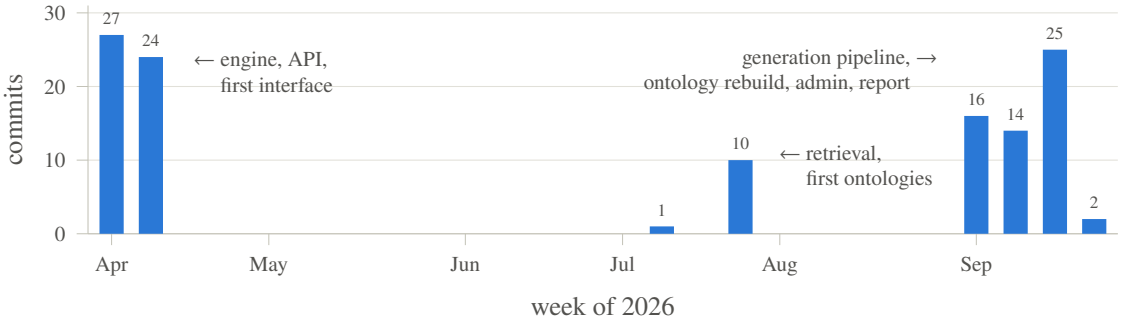


Figure 6.2: Version-control activity: 119 commits between 5 April and 26 September 2026. Work reached the shared repository in three bursts, so the schedule in Figure 6.1 is the plan of record rather than a trace of the commit history.

6.1.2 Budgeting and Resource Identification

The project was built entirely on free tiers and hardware the team already owned; direct monetary expenditure was zero. That is not the same as costless, so Table 6.2 itemises both the actual outlay and the notional value of what was consumed, and Table 6.3 models what a real deployment would cost.

The free tiers are a genuine constraint rather than a lucky break: the pipeline's key-pooling, adaptive backoff and resumable-progress design exist specifically because per-key rate limits, not money, were the scarce resource.

Table 6.1: Risk register: principal risks, their realisation, and mitigation

Risk	Realised?	Mitigation and outcome
Content generation infeasible at required scale	**Yes**	Measured early rather than assumed; switched to a hosted API with pooled keys. Cost the schedule, not the deliverable.
Knowledge structure inadequate for the inference built on it	**Yes**	Detected by structural analysis rather than by a user complaint; rebuild was affordable because the ontology build was automated from the start.
Machine-generated questions are wrong or off-topic	Partly	Two-model generate-then-verify with blind re-solve; only items passing both checks are written.
Source corpus corruption propagating silently	**Yes**	One malformed record destroyed 334 neighbours before the parser was made resynchronising; escape repair made unconditional.
External API rate limits or cost overrun	**Yes**	Key pooling, adaptive backoff, batch verification, resumable progress files. One parallelised run was halted mid-flight when its consumption rate proved unsustainable, and the work was restructured to a cheaper sequential method.
Knowledge loss across a five-person, multi-month project	Partly	A chain of handover documents recording state and rationale per subsystem; these were written precisely because context kept being lost between sessions and contributors.
Team member unavailability during examinations	Partly	Subsystem boundaries chosen so that no single absence blocked others.
Secret leakage into a public repository	No	Credentials confined to a git-ignored environment file from the first commit that needed one; never committed.

Table 6.2: Development resources: actual and notional cost

Resource	Basis	Actual cost
Engineering effort	5 students, one academic session, part-time alongside coursework	BDT 0 (in kind)
Development hardware	Personal laptops, CPU only, no GPU purchased	BDT 0 (owned)
Database and hosting	Supabase free tier	BDT 0
Model API for generation	Gemini free-tier keys, pooled across accounts to raise the rate ceiling	BDT 0
Embedding model	Open-weight multilingual model, run locally	BDT 0
Source content	Textbooks and question banks already owned by team members	BDT 0
Version control and CI	GitHub free tier	BDT 0
Total direct expenditure		**BDT 0**

Table 6.3: Indicative operating cost for a production deployment (order-of-magnitude estimate, not a quotation)

Item	Driver	Scale
One-time question generation	1,688 skills × 6 Bloom levels = 10,128 specifications, each a generation call plus a share of a batched verification call	Dominant one-time cost
Managed database	Tens of MB of content; one row per learner per skill	Lowest paid tier adequate at pilot scale
Application hosting	One small always-on process; inference is constant-time per response	Smallest instance class
Content refresh	Regeneration only when the syllabus or ontology changes	Infrequent
Domain expert review	Chemist and physicist review of the ontology (Section 5.2.6)	The largest genuinely unavoidable human cost

6.1.3 Economic Analysis and Financial Prospecting

The cost structure has one useful property: it is almost entirely fixed. Content generation and ontology construction are one-time costs; serving an additional learner costs a database row and a few constant-time updates. The marginal cost per learner is therefore close to zero, which is the necessary condition for any viable offering in this market, because Section 2.3 established that individual willingness to pay for software is low even though willingness to pay for preparation is high.

Three revenue paths follow from that, in descending order of realism:

1. **Institutional licensing.** Coaching centres and schools pay per cohort for diagnostic reporting that they cannot produce manually. The buyer has budget, the value proposition is diagnosis at a granularity a human cannot reach across a large class, and the number of buyers is small enough to reach directly.
2. **Freemium to learners.** Diagnostic free, unlimited practice paid. This fits observed behaviour but yields little per user and depends on scale the project does not have.
3. **Public or NGO deployment.** Offered at no cost as an equity intervention (Section 6.5.1), funded by a body whose objective is access rather than revenue. This is the path most consistent with the system's social value, and the least dependent on a commercial market.

The honest financial reading is that this is not, in its current form, a business: it is a prototype whose economics work only after the one-time content cost is paid and whose evidence of efficacy

— the thing an institution would actually buy — does not yet exist (Section 5.2.6). Establishing that evidence is a prerequisite for any of the three paths above, not a follow-on.

6.1.4 Sustainability in Business and Commercialisation

For the system to outlive the course, three things must be true. First, the knowledge structure must be maintainable by someone who is not the original team: this is addressed architecturally, since the ontology is compiled from a source file by a documented one-command build, with editorial decisions isolated in a single configuration file. Second, the content must be regenerable rather than hand-curated, which the pipeline provides. Third, the system must be transferable — which is why the repository carries an operational guide and a chain of handover documents written for a reader with no prior context, and why the test harness runs without credentials.

The realistic continuation path is academic rather than commercial: the codebase, the validated ontology and the documented open questions form a defined starting point for a subsequent capstone or thesis, with the domain expert review and a controlled efficacy study as the natural next pieces of work. A commercial path would require a sponsor to fund the generation run and the expert review before any user-facing launch.

6.2 Individual and Teamwork

6.2.1 Participation and Contribution

Ownership was primary rather than exclusive — the ontology rebuild in particular drew in contributors from outside its owning area, because it sat on every other subsystem's critical path.

6.2.2 Collaboration and Conflict Resolution

Collaboration was organised around the repository. Work proceeded on subsystem-aligned branches merged into a main line, so that the integration point was explicit and reviewable rather than continuous. The credential-free test harness served as the shared definition of "not broken": any contributor could run it in seconds before merging, which removed most integration disagreements by making them checkable rather than arguable.

The substantive technical disagreements in the project were about the ontology — specifically about granularity, since finer skills improve diagnosis but multiply content cost, and different contributors weighted those differently. These were resolved by writing the criteria down and testing them against data: the decision to rebuild was settled by the structural analysis in

Table 6.4: Contribution by subsystem

Member	Primary area	Contributions
Sayaad Muzahid Masfi (2105066)	Adaptive engine	BKT implementation, the three-phase mastery policy, the skill-graph data structure with its cycle check, and the policy specification document that the code is audited against.
Aurchi Chowdhury (2105083)	Backend API & sessions	FastAPI service, session lifecycle for diagnostic and practice, the tiered question search, and the spillover rules.
Omar Abdur Razzaque (2105094)	Content pipeline	Corpus parsing and repair, retrieval index, the generate-then-verify generation pipeline, and the database loader with its validation pass.
Sakif Naieb Raiyan (2105065)	Frontend	React client, the question-answer flow, mastery visualisation, bilingual interface, and mathematical rendering.
Rubaiyat -E-Zaman (2105104)	Ontology	Skill extraction from the corpus, the ontology compiler, the full-corpus rebuild and its editorial pass, and the structural validators.

Investigation 2 rather than by argument, and the granularity target was fixed by reference to what the gating policy needs to function. The general pattern the team converged on was to convert a disagreement about judgement into a measurement wherever one could be defined.

6.2.3 Leadership and Direction

Direction was distributed by subsystem rather than centralised: each area's owner set its technical direction and was accountable for it, with cross-cutting decisions — the model family, the granularity target, the decision to rebuild — taken jointly. The mechanism that kept direction coherent across a five-and-a-half-month project was documentary: each significant change was accompanied by a handover note recording not just what changed but why, so that a contributor returning to an area after weeks away, or a new contributor entering it, inherited the reasoning rather than only the code.

The clearest instance of planning after a setback is Investigation 1. When local generation was measured and found unable to finish within the schedule, the response was not to persist but to re-plan: the approach was abandoned, the alternative adopted, and the reasoning recorded so that the abandoned path would not be re-attempted by someone who had not seen the measurements.

6.2.4 Multidisciplinary Engagement

The project is multidisciplinary by construction: it required computer science (inference, graph algorithms, systems), educational psychology (knowledge tracing, Bloom's taxonomy, the zone of proximal development), and subject-matter expertise in three natural sciences at HSC level. The team's engagement outside its own discipline took three forms: reading and applying the education research literature rather than treating pedagogy as common sense; working directly with HSC Mathematics, Physics and Chemistry source material to build the ontology; and explicitly identifying the boundary of its own competence — the decision to mark 32 graph fragments and a body of authored chemistry edges as requiring a chemist's and a physicist's review, rather than resolving them by a computer scientist's intuition, is the clearest example of recognising where another discipline's authority was needed.

6.3 Impact on Society

Positive: access. The dominant positive impact is distributional. Expert diagnostic attention is currently rationed by money and geography — the best coaching is concentrated in cities and priced accordingly. A system whose marginal cost per learner is near zero can offer a form of that attention to a student in a district town with a phone and a connection. This is the project's principal social justification.

Positive: efficiency of effort. Redirecting a student from repeated failure on a downstream topic to the upstream skill actually missing reduces wasted study time in a year where time is the binding constraint.

Negative: misdiagnosis. The corresponding harm is specific and worth stating plainly. A wrong prerequisite edge or a miscalibrated parameter can send a learner to study something they already know, or release them from something they do not. In a high-stakes year that is a real cost borne by someone with no way to detect the error. The design response is the conservative mastery threshold and the refusal to guess prerequisite edges mechanically (Investigation 3); the honest response is that this risk is why efficacy evidence is a precondition for deployment, not a nicety.

Negative: intensification. A system that always has a next question can extend an already unhealthy preparation culture. The design does not currently counter this — there are no session limits or break prompts — and adding them is listed in future work.

Health and safety. No physical risk. The relevant health dimension is psychological: adaptive systems that target a constant challenge level deliberately keep the learner near their failure boundary, which is motivationally corrosive if presented as bare failure. Mitigation in the current design is that mastery is shown as a graph of what has been secured, not only as what remains.

Cultural. Bangla-first content with correct mathematical typesetting is a deliberate cultural position: it refuses the common assumption that serious technical study must happen in English, and it serves students whose mathematical vocabulary is Bangla.

Legal. Two obligations govern deployment: personal data of a predominantly under-20 user base, and copyright in source material. Both are addressed in Section 3.2; the current prototype's lack of authentication means it is not yet lawful to deploy publicly with real learner data, which is stated as a blocking condition rather than a limitation.

6.4 Environmental Impact and Life Cycle Sustainability

Development. Modest: five laptops over one session, and CPU-bound embedding of a corpus of a few thousand records. The abandoned local-inference approach would have been materially worse — weeks of continuous CPU computation — so the decision taken on schedule grounds in Investigation 1 also reduced energy consumption.

Content generation. This is the project's environmental hotspot. Large language model inference carries a real energy and water footprint in the provider's data centre, and the specification space at rebuild scale is 10,128 generation calls plus batched verification. Three design choices reduce it, all originally made for cost or throughput reasons: verification uses a smaller model than generation; verification calls are batched across several specifications; and progress is checkpointed so an interrupted run resumes rather than restarting. A fourth mitigation is intrinsic — generation is a one-time cost, not a per-learner one, and it amortises over every learner thereafter.

Operation. Very low. One small always-on process and a managed database; inference per response is arithmetic, not model inference. The system does no work while a learner is reading a question.

Retirement. Two obligations. Learner data must be deletable, which the schema supports through cascading deletion from the learner record, and which a production deployment should expose as an explicit account-deletion path. The generated question bank and ontology are the project's durable artefacts and are worth preserving beyond the system itself — as a syllabus-aligned skill graph they have value independent of this application, and releasing them rather than discarding them is the sustainable end-of-life choice.

6.5 Ethics

6.5.1 Equity and Inclusivity

Language. The system is Bangla-first for content, with a bilingual interface. This is the single largest inclusivity decision in the project: it admits students whose technical vocabulary is Bangla, who are systematically disadvantaged by English-only tools.

Cost. Zero marginal cost per learner makes free provision technically possible, which is the precondition for reaching students for whom coaching fees are prohibitive.

Access assumptions, and who is still excluded. The design assumes a browser and a connection. That excludes students without either — which is precisely the group the access argument above claims to serve. This tension is not resolved in the current system. Offline capability and low-bandwidth operation are the honest mitigations, and neither is implemented.

Disability. Mathematics is rendered as accessible markup rather than as images, and controls are keyboard-operable, but no formal accessibility audit against WCAG AA [21] was performed and no screen-reader testing was done. Stating this as an outstanding gap is more useful than claiming compliance that has not been verified.

Fairness across learners. One model-level equity issue deserves naming: the parameters in Table 4.2 are uniform across all learners. A learner whose true guess or slip behaviour differs from those priors — for instance, a careful learner who slips rarely — is modelled slightly wrongly, and uniformly. Individualised parameters [5] are the known remedy and require response data the system does not yet have.

6.5.2 Accountability and Personal Responsibility

Responsibility for the system's behaviour rests with the team, and within the team it is traceable: subsystem ownership is recorded (Table 6.4) and every change is attributable through version control. The project's accountability practices were concrete rather than declarative — the defect in Section 5.3 was reported, analysed and guarded against rather than quietly fixed, and the two over-flagging validators in Investigation 4 were documented as team errors of judgement rather than deleted.

The accountability commitment that matters most for this system is about claims. The team is responsible for not asserting that ATLAS improves examination outcomes, because no evidence for that exists. That commitment is why Section 5.2.6 exists in this report in the form it does.

6.5.3 Proper Use of Intellectual Property

Prior art and literature. The model family, the taxonomy and the retrieval approach are not original to this project and are cited to their sources [2,3,11,13,14,16]. The original contribution claimed is the composition — Bloom-conditioned BKT with ancestor pull-up and conjunctive gating over a syllabus-derived graph — not its components.

Third-party software. All third-party components are open-source under permissive licences and are used as dependencies rather than copied into the codebase: FastAPI, Uvicorn, Pydantic (backend); React, Vite, Tailwind CSS, KaTeX, Lucide (frontend); sentence-transformers and NumPy (pipeline); PostgreSQL via Supabase. Their licences are carried in the dependency manifests.

Source content. The corpus is drawn from copyrighted HSC textbooks and published question banks. It is used as a private retrieval and grounding corpus; it is not redistributed, is excluded from distributable artefacts, and generated items are new text rather than reproductions. Any public deployment would require this position to be reviewed with the rights holders.

Generative AI disclosure. Generative AI was used substantively in this project and is disclosed accordingly:

- **As a system component.** The question bank is machine-generated via a hosted large language model, with a second model verifying each item. This is not an incidental use — it is a designed part of the architecture (Section 4.2.9), and the verification stage exists precisely because unverified model output is not trustworthy as assessment content.
- **As a development tool.** AI coding assistants were used during development for code generation, refactoring, documentation, the ontology extraction and consolidation passes, and in the preparation of this report. All such output was reviewed by team members before being committed, and the team accepts full responsibility for it (Section 6.5.2). Investigation 4 is a direct record of that review process catching machine-produced errors — both over-flagging validators were AI-assisted artefacts corrected by hand review.

6.5.4 Professionalism and Ethical Codes

The ACM Code of Ethics [23] and the IEEE Code of Ethics [22] both place avoiding harm and honest representation of work above the practitioner's own interest. Three project decisions follow directly from those obligations and were in tension with the team's interest in presenting a polished result:

1. **Refusing an unevidenced efficacy claim** (ACM 1.3, "be honest and trustworthy"). The straightforward way to strengthen this report would be to assert that the system improves

learning. Section 5.2.6 instead states that no such evidence was gathered.

2. **Declining to connect prerequisite edges mechanically** (ACM 1.2, "avoid harm"). Accepting all 83 machine proposals would have produced a better-looking connectivity statistic. Thirty-five were rejected because a wrong edge silently corrupts a learner's diagnosis, which is a harm the statistic would have concealed.
3. **Documenting defects rather than quietly closing them** (IEEE 7.8.7, accepting and offering honest criticism of technical work). The topic-code defect, the corpus corruption and the over-flagging validators are all recorded in the repository and in this report with their causes.

The public-interest dimension specific to this system is that its users are young people making consequential decisions under pressure, largely without the expertise to audit the advice they are given. That asymmetry is the reason the team treats "do not claim more than the evidence supports" as the project's governing professional obligation rather than as a reporting formality.

Chapter 7

Conclusion

7.1 Conclusion and Future Work

7.1.1 Achievement against objectives

Table 7.1: Achievement against the objectives of Section 1.4

Obj.	Status	Basis
O1	Achieved	1,688 skills, 1,743 edges, all 183 syllabus topics populated; validated acyclic, depth 9, zero isolated skills, zero unresolved references; adopted as the served catalogue
O2	Achieved	Bloom-conditioned BKT implemented from the equations; exercised on every response by the verification harness
O3	Achieved in implementation	Ancestor pull-up and conjunctive gating implemented; pull-up verified after every correct answer, but the ordering is not kept through later wrong answers (4–8% of edges invert). Whether the estimates match real learner knowledge is untested
O4	Achieved	The diagnostic stays inside the section under test, builds on stored mastery, runs to completion in all 51 sections and releases the mastery gate; eight sections end short of their advertised length
O5	Achieved	Topic practice runs to the mastery threshold; spillover fires on the intended evidence, 84 times in the live check
O6	**Largely achieved**	12,223 verified questions on the served skills; 9,037 of the $1{,}688 \times 6 = 10{,}128$ specifications (89%) covered, Physics 74%; 41 skills without a question; 8.3% of served questions have rendering defects
O7	Achieved	Ten-route web client (eight learner routes, two administrator) with diagnostic, practice and mastery views, bilingual, with Bangla and LaTeX rendering
O8	Achieved	29-check credential-free harness passing completely, and a live end-to-end check across all 51 sections (Section 5.2.3)

The project set out to build a tutor that could tell a student *which* foundation is missing rather than merely that they are wrong, and the mechanism for that — an interpretable per-skill model, propagating along an explicit prerequisite graph, gated conjunctively — is built and demonstrably

self-consistent. The single most valuable thing the project learned is not in the engine at all: it is that the knowledge structure, not the inference algorithm, is the binding constraint on a system like this. A sophisticated update rule has almost nothing to reason with over a graph that holds fewer than four skills per topic and leaves many skills with no prerequisite at all; what fixed it was coverage and connectedness, not a denser edge ratio. Most of the engineering effort in the second half of the project went into the ontology for exactly that reason, and that is the finding most likely to transfer to anyone attempting similar work.

7.1.2 Limitations

Stated plainly, and in the order that matters:

1. **No efficacy evidence.** The system has never been used by a real learner. Every claim about learning benefit is an argument from design.
2. **No fitted parameters.** All model parameters are pedagogical priors; the literature is clear that fitted and individualised parameters do better [4, 5].
3. **Question-bank quality now limits more than its size.** The generation run covers 89% of the specification space, but Physics lags at 74%, 41 skills have no question, 8.3% of served questions render wrongly, and 7,692 legacy-keyed questions are unreachable.
4. **Open engine defects found by the end-to-end check** (Section 5.3): the ancestor ordering is not kept through wrong answers; fallback items are credited to the target skill and level; some diagnostics end early; one section's diagnostic can unlock another; and question selection has a long latency tail in thin sections.
5. **The ontology has not been reviewed by domain experts** — 32 disconnected fragments and a body of hand-authored chemistry edges await a chemist and a physicist.
6. **Production-readiness gaps:** no password authentication (the administrator pages trust a username header), in-memory session state, wide-open CORS, no accessibility audit, no load test.

7.1.3 Future work

Immediate, and blocking deployment. Password authentication and server-side sessions; persistent session state; CORS restriction. These are small, well-understood, and are the difference between a demonstrator and something that may lawfully hold real learner data.

Closing the engine defects the end-to-end check found. Credit each answer to the skill and Bloom level of the item actually served; record diagnostic completion per learner and section

instead of inferring it from any stored mastery; size each diagnostic by the skills that can be asked about; bound the fallback ladder's queries; and decide how a wrong answer on an ancestor should interact with the ordering its descendants imply (Section 5.3).

Completing the knowledge base. The generation run has covered 89% of the $1{,}688 \times 6 = 10{,}128$ specifications; what remains is the other 1,091, Physics first, and re-keying or regenerating the 7,692 legacy questions. The bank also needs a repair pass for the 1,015 questions that render wrongly, and its distractor tags need pointing at served skills, since 38% still name legacy ones. Alongside it, domain-expert review of the 32 disconnected fragments and the hand-authored edges, since a wrong prerequisite edge propagates silently under Equation (4.4).

Establishing efficacy — the most important item. A controlled study with real candidates: does the system's mastery estimate predict performance on held-out questions better than a baseline, measured by AUC as the knowledge tracing literature does; and does directing study by the system's diagnosis produce better outcomes than self-directed study? Until this is done, the central claim of the project is unverified.

Model improvements once data exists. Fitted and individualised BKT parameters [5]; contextual estimation of slip and guess [4]; and, with sufficient data, a comparison against a deep knowledge tracing baseline [10]. The engine was deliberately built with a stateless, replaceable model interface so that this substitution requires no change to its call sites.

Pedagogical extensions. Forgetting/decay in the mastery estimate, which the current model omits entirely and which matters over a months-long preparation period [18]; spaced review scheduling; and session pacing that resists the intensification risk noted in Section 6.3.

7.2 Continuous Engagement and Self-directed Learning

Little of what this project required was covered by prior coursework, and the learning it forced was mostly self-directed.

Domain literature learned independently. The knowledge-tracing literature was new to the team: the original BKT formulation [2], the variants that refine its noise parameters and individualise them [4–6], the competing model families [8–10], the prerequisite-structured student models closest to this design [7], and the older knowledge-space theory that supplies the graph formulation [11, 12]. The educational side — the revised Bloom taxonomy [3] and the zone of proximal development [14] — was read as source material for design decisions rather than as background, since both appear directly in the implemented policy.

Emerging technical practice. Retrieval-augmented generation [16] and multilingual sentence embeddings [17] were adopted from current practice; the survey literature on automatic question generation [15] was what convinced the team that generation without an independent verification

stage would not produce usable assessment items, which directly shaped Section 4.2.9.

Tools and technologies learned during the project. FastAPI and Pydantic; React with Vite and Tailwind; Supabase and its constraint model; KaTeX and the specific difficulties of mixing Bangla text with LaTeX mathematics; local and hosted large-language-model inference including quantised local serving, and the practical engineering around hosted APIs — rate limiting, key pooling, adaptive backoff and resumable batch jobs.

Learning from failure, which was the majority of it. Three episodes taught more than any tutorial. Measuring local inference before committing to it would have saved weeks — the lesson being to measure the throughput of a critical dependency before building on it. The silent JSON escape corruption taught that a parser accepting input is not evidence the input was understood, and that data corruption which raises no error is the expensive kind. The two over-flagging validators taught that a heuristic must be calibrated against hand review before its output is treated as a defect list.

Practices adopted. The habit that most changed how the team works is documentary: writing down the reasoning behind a decision at the moment it is taken, not afterwards. On a five-person project spanning months, the recurring bottleneck was not writing code but recovering context — why a constant has the value it has, why an approach was abandoned, what a validator's output actually means. The chain of handover documents in the repository exists because of that, and it is the practice the team expects to carry into work beyond this course.

References

[1] K. VanLehn, "The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems," *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011.

[2] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253–278, 1995.

[3] L. W. Anderson and D. R. Krathwohl, *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman, 2001.

[4] R. S. J. d. Baker, A. T. Corbett, and V. Aleven, "More accurate student modeling through contextual estimation of slip and guess probabilities in Bayesian knowledge tracing," in *Intelligent Tutoring Systems (ITS)*, 2008, pp. 406–415.

[5] M. V. Yudelson, K. R. Koedinger, and G. J. Gordon, "Individualized Bayesian knowledge tracing models," in *Artificial Intelligence in Education (AIED)*, 2013, pp. 171–180.

[6] Z. A. Pardos and N. T. Heffernan, "KT-IDEM: Introducing item difficulty to the knowledge tracing model," in *User Modeling, Adaptation and Personalization (UMAP)*, 2011, pp. 243–254.

[7] T. Käser, S. Klingler, A. G. Schwing, and M. Gross, "Dynamic Bayesian networks for student modeling," *IEEE Transactions on Learning Technologies*, vol. 10, no. 4, pp. 450–462, 2017.

[8] H. Cen, K. Koedinger, and B. Junker, "Learning factors analysis – a general method for cognitive model evaluation and improvement," in *Intelligent Tutoring Systems (ITS)*, 2006, pp. 164–175.

[9] P. I. Pavlik, H. Cen, and K. R. Koedinger, "Performance factors analysis – a new alternative to knowledge tracing," in *Artificial Intelligence in Education (AIED)*, 2009, pp. 531–538.

[10] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. J. Guibas, and J. Sohl-Dickstein, "Deep knowledge tracing," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 505–513.

[11] J.-P. Doignon and J.-C. Falmagne, "Spaces for the assessment of knowledge," *International Journal of Man-Machine Studies*, vol. 23, no. 2, pp. 175–196, 1985.

[12] J.-C. Falmagne and J.-P. Doignon, *Learning Spaces: Interdisciplinary Applied Mathematics*. Springer, 2011.

[13] B. S. Bloom, *Taxonomy of Educational Objectives: The Classification of Educational Goals, Handbook I: Cognitive Domain*. David McKay Company, 1956.

[14] L. S. Vygotsky, *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, 1978.

[15] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, "A systematic review of automatic question generation for educational purposes," *International Journal of Artificial Intelligence in Education*, vol. 30, no. 1, pp. 121–204, 2020.

[16] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[17] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," *Proceedings of EMNLP-IJCNLP*, pp. 3982–3992, 2019.

[18] H. Ebbinghaus, *Memory: A Contribution to Experimental Psychology*. Teachers College, Columbia University, 1885.

[19] Government of the People's Republic of Bangladesh, "Digital security and data protection policy framework," 2023.

[20] OWASP Foundation, "OWASP top 10: The ten most critical web application security risks," 2021.

[21] World Wide Web Consortium, "Web content accessibility guidelines (WCAG) 2.1," 2018, W3C Recommendation.

[22] Institute of Electrical and Electronics Engineers, "IEEE code of ethics," 2020.

[23] Association for Computing Machinery, "ACM code of ethics and professional conduct," 2018.